

Notification Platform — Engineering Design Proposal

Status: Draft for senior-engineering review · **Author:** Platform Engineering · **Date:** June 2026

How to read this document. This is a synthesis of nine production engineering write-ups (Uber RAMEN/gRPC, Netflix RENO, the public RAMEN-vs-RENO comparison, Uber's Mobile Content Delivery feed, Uber's ML push-timing "CCG" system, Slack's notification tracing, Airbnb's OMNI, Razorpay's webhook-scaling story, and two reference system-design guides) plus an internal Kafka-based trading-notification draft. Every major decision is tied back to evidence from one or more of those systems, and where companies disagree, the trade-off is spelled out rather than resolved arbitrarily. Recommendations I add from general distributed-systems practice that are **not** drawn from the source material are explicitly tagged [**Independent recommendation**].

Source key (used throughout for attribution):

Tag	System	One-line summary
Uber-RAMEN	Uber Real-Time Push Platform	Polling → SSE push → gRPC; Fireball + gateway + delivery separation
Netflix-RENO	Netflix Rapid Event Notification System	Manhattan single source, priority SQS, hybrid push+pull
Cmp	RAMEN vs RENO comparison	Side-by-side of the two above
Uber-Feed	Uber Mobile Content Delivery	Fan-out-on-write vs read; Cassandra feed store
Uber-CCG	Uber ML Push Timing (Consumer Comms Gateway)	Per-user inbox + ILP scheduler + XGBoost timing
Slack-Trace	Slack Tracing Notifications	notification_id as trace_id, 100% sampling, span links
Airbnb-OMNI	Airbnb Promotions & Communications	Audience/Workflow/Optimization/Presentation/Rendering/Delivery services
Razorpay	Razorpay Notification Service scaling	Async DB writes via stream, P0/P1/P2 queues, QoS for slow consumers
Ref-AM / Ref-MB	AlgoMaster & Madhur Banger guides	Capacity math, idempotency/backoff patterns, broker comparison
Internal-Draft	Internal Kafka trading-notification doc	Topic taxonomy, tiered retry, Redis scheduler, fail-open/closed preferences

1. Executive Summary

1.1 What we are building

A multi-channel notification platform (email, SMS, push, in-app/WebSocket, with WhatsApp and webhooks as pluggable later channels) that reliably delivers **transactional, informational, and promotional** notifications to millions of users. The platform has **two entry points** — a synchronous client API for services that want to send a notification, and an **event-driven ingestion path** that auto-generates notifications from a stream of domain events (in our context, market-data ticks crossing user-defined alerts). This dual-entry model comes directly from **Internal-Draft**, and it mirrors the producer taxonomy in **Ref-MB** (user actions, system events, scheduled jobs, external integrations) and Uber's **Fireball** trigger model in **Uber-RAMEN**.

1.2 Recommended architecture in one paragraph

An **event-driven, queue-decoupled pipeline** built on **Apache Kafka** as the backbone, with a thin stateless **Notification API / ingestion service**, an **enrichment + routing stage**, **per-channel worker fleets**, and **channel adapters** to external providers (SES/SendGrid, Twilio, FCM/APNs, a WebSocket gateway). **PostgreSQL** holds canonical relational state (contacts, preferences, subscriptions, templates, device tokens); **Redis** handles the preference cache, idempotency keys, the scheduling timer-queue, and rate limits; a **wide-column store** (**Cassandra/ScyllaDB**) holds the high-volume delivery log once it outgrows Postgres; and **ClickHouse/Elasticsearch** powers analytics and search. Reliability rests on **at-least-once delivery + idempotency keys (effectively-once)**, a **tiered retry ladder → DLQ**, **per-user partition ordering**, **bulkheaded per-channel delivery**, and **priority queues with rate limiting**. Observability is **end-to-end tracing modeled on Slack's notification-as-a-trace approach**, plus Prometheus/Grafana metrics.

1.3 The five takeaways that shaped the design

1. **Polling does not scale; go event-driven.** Uber found that at peak, *80% of API-gateway requests were polling calls* (**Uber-RAMEN**), driving battery drain, app sluggishness and network congestion, and making cold-start the worst case. The entire industry corpus here is a migration story away from polling toward server-initiated, event-driven delivery.
2. **Decouple triggering, creation, and delivery.** Uber's clean split between *Fireball* (when to push), *API gateway* (what to push), and the *delivery system* (how to push) is the reason they could later swap SSE for gRPC without touching business logic (**Uber-RAMEN**). Airbnb's OMNI is the same idea taken further: Audience/Workflow/Optimization/Presentation/Rendering/Delivery as separate services (**Airbnb-OMNI**).
3. **Protect the datastore — it is the first thing to fall over.** Razorpay hit a hard wall scaling worker pods because each pod added DB IOPS; vertical scaling (2x→4x→8x.large) bought time but eventually "was not helping anymore." Their fix —

writing to the DB asynchronously through a stream so write rate is decoupled from worker count — is one of the most reusable lessons in the whole corpus (**Razorpay**).

4. **Priority + isolation are mandatory, not optional.** Netflix shards traffic into priority queues and clusters (**Netflix-RENO**); Razorpay uses P0/P1/P2 queues plus a rate-limit queue and per-consumer QoS so one "rogue event" or one slow customer cannot starve everyone else (**Razorpay**). A margin call must never queue behind a marketing digest.
5. **You cannot operate what you cannot trace.** Slack's notification debugging took *multi-day cross-team investigations* until they modeled each notification as its own trace; that cut customer-issue triage time by **30%** (**Slack-Trace**). Observability must be designed in from day one — Airbnb explicitly lists "content governance and guardrails should have been mandatory from the start" as a top regret (**Airbnb-OMNI**).

1.4 Important lessons learned from the companies (condensed)

Company	The expensive lesson	What we do about it
Uber	Ringpop's gossip-based decentralized sharding stopped converging as the ring grew; Node.js single-thread event-loop lag caused message loss	Use centralized, battle-tested coordination (ZooKeeper/Helix-style) for any stateful connection sharding; keep hot path off single-threaded runtimes
Uber	SSE was unidirectional, text-only, with delayed acks → unreliable acknowledgements	If we build persistent in-app connections, use a bidirectional binary protocol (gRPC/WebSocket) with instant acks
Netflix	Smart TVs are offline most of the day; push-only loses messages	Hybrid push and pull; persist notifications so devices can poll at their own cadence
Airbnb	ElasticSearch silently bloated with write-once attributes	Allowlist indexed fields, keep schema sparse, separate batch vs realtime fields
Razorpay	Synchronous DB writes coupled write throughput to worker count	Async DB writes via a stream; throttle IOPS independently of workers
Slack	Fragmented per-system logging made notifications un-debuggable	One trace per notification, 100% sampled, span-linked to the originating event

2. Functional Requirements

The list below is the union of every functional capability mentioned across the sources, mapped to its origin. "Phase" indicates when we build it (see §23).

#	Requirement	Description	Evidence	Phase
F1	Send notification (sync API)	A client service calls a REST/gRPC endpoint to send a notification	Ref-AM, Ref-MB, Internal-Draft	MVP
F2	Event-driven send	Auto-generate notifications from a domain-event stream (e.g., market tick crosses an alert)	Internal-Draft, Uber-RAMEN (Fireball), Uber-Feed	MVP
F3	Multi-channel	Email, SMS, push, in-app behind a common adapter interface	All sources	MVP
F4	WebSocket / real-time in-app	Persistent connections for live in-app delivery + unread counts	Ref-MB, Netflix-RENO (Zuul Push), Uber-RAMEN	P2
F5	WhatsApp / extensible channels	Pluggable new channels without core changes	Airbnb-OMNI (reachability), Internal-Draft	P3
F6	Scheduled notifications	Send at a specified future time	Ref-AM, Uber-CCG, Airbnb-OMNI (hourly cron), Internal-Draft	MVP
F7	Delayed notifications	Relative delay ("in 10 minutes")	Internal-Draft (Redis ZSET), Uber-CCG	MVP
F8	Recurring / digest	Batched daily/weekly digests	Ref-MB (bulk vs real-time), Uber-Feed (bulk)	P3
F9	Retry	Retry transient failures with backoff	All sources	MVP
F10	Dead-letter handling	Exhausted retries isolated for inspection/replay	Internal-Draft, Ref-AM, Ref-MB	MVP
F11	Read / Unread state	Track and expose read state for in-app	Ref-MB, Netflix-RENO (My List/poll)	P2
F12	Notification preferences	Per-channel, per-category opt-in/out	All sources	MVP
F13	User opt-out / consent	Honor opt-out; keep consent audit trail	Ref-MB (GDPR/CAN-SPAM/TCPA), Airbnb-OMNI	MVP

#	Requirement	Description	Evidence	Phase
F14	Quiet hours	Suppress non-critical during user quiet hours	Ref-MB, Internal-Draft	P2
F15	Priority notifications	High/Medium/Low (or P0/P1/P2) with differentiated handling	Uber-RAMEN, Netflix-RENO, Razorpay	MVP
F16	Templates	Reusable, versioned message templates per channel/locale	Ref-AM, Airbnb-OMNI (Presentation/Rendering), Uber-Feed (JSON blocks)	MVP
F17	Personalization	Variable substitution, user context, propensity/ranking	Ref-MB, Airbnb-OMNI (Optimization), Uber-CCG (XGBoost)	P2/P3
F18	Multi-language	Locale-aware rendering and translation	Airbnb-OMNI (Translation Service), Uber-Feed	P2
F19	Notification history	Durable per-user history users can query	Ref-AM, Netflix-RENO (Cassandra poll store), Uber-Feed	P2
F20	Bulk notifications	Send to large cohorts (tens of millions)	Airbnb-OMNI, Uber-Feed (Hive/Spark)	P2
F21	Campaign notifications	Self-service promotional campaigns w/ lifecycle + approval	Airbnb-OMNI (Campaign Service + UI)	P3
F22	Transactional notifications	Real-time, action-triggered (OTP, order filled)	Airbnb-OMNI, Ref-AM, Internal-Draft	MVP
F23	Send-time optimization	ML-chosen optimal delivery time + frequency capping	Uber-CCG (ILP solver), Airbnb-OMNI	P3
F24	Frequency capping / dedup of competing messages	Don't overwhelm or send conflicting messages	Uber-CCG, Ref-AM (rate limit)	P2
F25	Analytics	Throughput, open rates, funnel, per-channel success	Slack-Trace, Airbnb-OMNI, Razorpay	P2

#	Requirement	Description	Evidence	Phase
F26	Delivery tracking	Per-notification delivery status & timestamps	Slack-Trace, Ref-AM (notification_logs)	MVP
F27	Failure tracking	Capture failure reasons, provider errors	Razorpay, Slack-Trace, Ref-AM	MVP
F28	Idempotency	Caller-supplied idempotency key; no duplicate sends	Ref-MB, Internal-Draft	MVP
F29	Deduplication	Merge duplicate generated events	Netflix-RENO, Uber-RAMEN	MVP
F30	Fallback channels	Escalate critical message to another channel on failure	Ref-MB, Internal-Draft (category logic)	P2
F31	Targeted delivery	Notify only specific device categories / online devices	Netflix-RENO, Uber-Feed (targeting)	P2
F32	Audience segmentation	Build cohorts from user attributes	Airbnb-OMNI (Audience Service)	P3
F33	Multi-tenancy	Per-tenant routing, provider settings, isolation	Ref-MB (tenant_id)	P3
F34	Self-service admin/console	Create/manage campaigns, view dashboards, replay DLQ	Airbnb-OMNI, Razorpay (Grafana)	P2/P3
F35	Device token management	Store, rotate, and invalidate stale push tokens	Ref-MB, Uber-RAMEN (device context)	MVP
F36	Webhook notifications	Outbound HTTP callbacks to subscriber endpoints	Razorpay	P2

3. Non-Functional Requirements

These are the engineering constraints, with concrete targets and the evidence behind each number.

3.1 Targets at a glance (SLOs)

Dimension	Target	Rationale / evidence
Availability	99.99% for the ingestion + delivery control plane	Airbnb-OMNI states a 99.99% uptime goal; Uber reports 99.99% server-side reliability for its push infra
Delivery success rate (SLO)	$\geq 99\%$ transactional; $\geq 90\%$ across all channels including promo	Airbnb-OMNI targets 90% delivery success across all channels
End-to-end latency (p99)	< 5 s transactional API→provider handoff; < 30 s for the full pipeline incl. heavy enrichment	Airbnb-OMNI's Delivery Service achieves < 30 s end-to-end; Razorpay's p99 was ~ 2 s healthy, degrading to ~ 4 s under load
Critical-path latency (OTP/margin call)	< 2 s to provider	Ref-MB ("OTPs in seconds"); fintech regulatory expectation in Internal-Draft
Throughput	Sustain 17k notifications/sec, absorb bursts to 150k+ events/sec	Ref-AM computes ~ 17 k/s at 50M users; Netflix-RENO peaks at 150k events/sec
Error budget	0.01% unavailability/month ≈ 4.3 min; failed-delivery alert threshold 5% per provider	Ref-MB example alert "SMS provider failure rate $> 5\%$ "

3.2 The full constraint list

- **Scalability (horizontal).** Every tier must scale by adding instances, not by growing one box. Workers are **stateless** and join Kafka **consumer groups** (Ref-MB, Ref-AM). This is the explicit antidote to Razorpay's vertical-DB-scaling cliff and Uber's Ringpop scaling limits.
- **Throughput & QPS.** See §4 capacity model. Design point: 250M notifications/day steady, 1M/min peaks.
- **Durability.** No silent message loss. Kafka **replication factor** ≥ 3 , producer **acks=all**, schema validation at ingestion (Internal-Draft). Cassandra cross-region replication for history (Uber-Feed, Netflix-RENO).
- **Consistency.** Strong consistency for preferences/consent (PostgreSQL); eventual consistency acceptable for analytics and delivery logs.
- **Delivery semantics.** **At-least-once + idempotency keys** → **effectively-once**. This is the consensus across Internal-Draft, Ref-MB, Ref-AM, and Uber-RAMEN ("at-least-

once guarantee"). Exactly-once is offered only where the queue supports it and the use case demands it (Ref-AM).

- **Event ordering. Per-user ordering** via Kafka partition key = `user_id` (Internal-Draft, Ref-MB, Uber-RAMEN sharded by user UUID). Global ordering is explicitly **not** a requirement — it would destroy parallelism.
- **Fault tolerance & failure isolation. Bulkheading**: a failing channel/provider must not block others. Netflix parallelizes delivery per platform "best-effort per platform" (Netflix-RENO); we do the same per channel. Razorpay's QoS degrades slow consumers rather than letting them block workers.
- **Backpressure handling**. Queues buffer producer/consumer rate mismatch; under sustained overload, **shed low-priority load** (Ref-MB "drop low-priority under high load"; Airbnb-OMNI backpressure + rate limiting to isolate spiky promo traffic).
- **Rate limiting**. Per-user, per-channel, per-tenant, and per-event-type. Razorpay's rate-limit queue is the model; also protects against DoS (Ref-AM, Ref-MB).
- **Reliability / graceful degradation**. Circuit breakers around providers; pre-warm provider connections before known spikes (Ref-MB). Fail-open for critical, fail-closed for marketing (Internal-Draft).
- **High availability & regional deployment**. Multi-AZ replication (Ref-AM); region-based clusters near user populations (Ref-MB); Cassandra data-center replication so users can be served during failover (Uber-Feed).
- **Disaster recovery**. Kafka retention + the audit topic enable **replay** for recovery/auditing (Internal-Draft). RPO target ≈ 0 for transactional (acks=all + RF3); RTO target < 15 min via multi-region standby. **[Independent recommendation]** for the explicit RPO/RTO numbers.
- **Security**. OAuth2/mTLS for API; RBAC for admin; encrypt PII in transit and at rest; secrets in a vault; webhook signing (Ref-MB, Ref-AM, Razorpay). See §18.
- **Compliance & data retention**. GDPR right-to-be-forgotten, CAN-SPAM/TCPA for SMS/email, consent audit trail (Ref-MB); fintech: regulated mandatory messages cannot be opted out of (Internal-Draft). Retention: ingest/enriched hours-days, audit weeks-months or log-compacted (Internal-Draft); archive old logs to cold storage (Ref-AM).
- **Observability**. Metrics + logs + tracing as first-class (all sources). Specific signals in §17.
- **Maintainability & extensibility**. New channels and new use cases must be "plug-and-play" — Netflix's explicit design win (Netflix-RENO); Airbnb's omni-channel adapter interface; Uber's protocol-agnostic delivery layer.
- **Cost efficiency**. SMS is the dominant variable cost (Airbnb spends "millions ... especially for SMS"). Batching, channel-shift, caching, and async writes all reduce cost — see §16.
- **Cold-start considerations**. Uber's worst case was app cold-start triggering many concurrent pulls; our event-driven push model plus a persisted history users poll on

open avoids the thundering-herd cold start (Uber-RAMEN, Netflix-RENO online-registry + poll store).

- **Capacity planning.** See §4.

4. Capacity Estimation

The methodology follows **Ref-AM** ("50M daily users $\times$ 5 notifications/day = 250M/day; 1M/min peak $\Rightarrow$ $\sim 17\text{k/s}$ ") and the sizing rule of thumb in **Internal-Draft** (*avg message size $\times$ QPS $\times$ retention $\times$ safety margin*). All numbers are order-of-magnitude planning figures, not guarantees.

4.1 Assumptions

- Avg notifications per active user per day: **5** (Ref-AM).
- Fan-out: avg **1.4 channels** per notification (some go to push+email).
- Avg event/message size on the wire: **1 KB** (Ref-AM uses 1 KB for both user and notification records).
- Peak-to-average ratio: $\sim 30\times$ for short bursts (Ref-AM's 1M/min peak vs $\sim 2.9\text{k/s}$ average at 250M/day implies a large multiplier; Netflix-RENO's 150k/s peak confirms big spikes are real).
- Delivery-log row: **$\sim 0.5\text{ KB}$** ; history retention: **90 days** hot, then archive.

4.2 Scaling table

Metric	100K users	1M users	10M users	100M users
Notifications/day	500K	5M	50M	500M
Avg notifications/sec	~ 6	~ 58	~ 580	$\sim 5,800$
Peak notifications/sec ($\sim 30\times$)	~ 175	$\sim 1,740$	$\sim 17,400$	$\sim 174,000$
Channel sends/sec (avg, $\times 1.4$)	~ 8	~ 81	~ 810	$\sim 8,100$
API writes/sec (avg)	~ 6	~ 58	~ 580	$\sim 5,800$
History reads/sec (in-app polls, $\sim 2\times$ sends)	~ 12	~ 116	$\sim 1,160$	$\sim 11,600$
Kafka ingress (peak, 1KB msgs)	$\sim 0.2\text{ MB/s}$	$\sim 1.7\text{ MB/s}$	$\sim 17\text{ MB/s}$	$\sim 170\text{ MB/s}$
Daily delivery-log growth (0.5KB/send)	$\sim 0.35\text{ GB}$	$\sim 3.5\text{ GB}$	$\sim 35\text{ GB}$	$\sim 350\text{ GB}$
90-day hot delivery log	$\sim 31\text{ GB}$	$\sim 315\text{ GB}$	$\sim 3.1\text{ TB}$	$\sim 31\text{ TB}$

Metric	100K users	1M users	10M users	100M users
User/preference data (1KB/user)	100 MB	1 GB	10 GB	100 GB
Channel worker pods (peak, ~500 sends/s/pod)	1-2	4-8	35-70	350-700
Kafka partitions/channel topic ($\approx$ target consumers)	6	12	48	192
Redis memory (dedupe keys, 7-day TTL, ~100B/key)	~0.3 GB	~3 GB	~30 GB	~300 GB (cluster)

Worked example — 100M users, peak

- Peak $\approx 500\text{M/day} \div 86,400 \times 30 \approx \mathbf{174k\ notifications/s}$. This is in the same range as Netflix-RENO's 150k/s and above Uber-RAMEN's 250k messages/s peak, so it is a proven-feasible regime — but only with horizontal scaling.
- Worker math: at ~500 sends/s/pod (a conservative figure given each send is an outbound network call with retries), $174k \times 1.4 \approx \mathbf{244k\ sends/s} \Rightarrow \mathbf{\sim 490\ pods\ at\ peak}$, autoscaled down to a fraction off-peak.
- Kafka: $170\text{ MB/s ingress} \times \text{RF3} = \sim 510\text{ MB/s replicated write}$; with 1-day retention on ingest/enriched that's $\sim 14.7\text{ TB/day}$ pre-replication of hot Kafka storage — which is why **per-topic retention must be short** (Internal-Draft) and the durable record lives in the compacted audit topic.

4.3 Scaling bottlenecks — the order they fall over

This ordering is **directly informed by Razorpay's experience** (the DB was always the first wall) and Uber's (the connection/topology layer was theirs):

1. **Relational DB write IOPS** — first to saturate. Razorpay hit this at $\sim 2\text{K TPS}$ and could not scale pods further without degrading the DB. **Mitigation: async writes via a stream (§9, §11).**
2. **Synchronous provider calls** — slow third-party endpoints block workers. Razorpay's QoS and Netflix's bulkheading address this.
3. **Hot partitions** — a celebrity user or a single instrument's alert fan-out skews one Kafka partition / one Cassandra row. Uber-Feed deliberately partitions by `user_id`; for fan-in events (one tick $\rightarrow$ many users) we fan out on the enrichment stage, not the partition.
4. **WebSocket connection memory** — persistent in-app connections are RAM-bound; Uber held 1.5M concurrent connections with Netty zero-copy buffers.
5. **ElasticSearch attribute bloat** — Airbnb's audience store strained under ever-growing indexed fields.

4.4 Cost implications (rough, cloud list-price order-of-magnitude)

Driver	Note	Source signal
SMS provider fees	Dominant cost; can be millions/yr at scale	Airbnb-OMNI
Kafka storage/throughput	Controlled by retention; keep ingest short	Internal-Draft
DB IOPS	Async writes cut this dramatically	Razorpay
Egress/bandwidth	Push/email cheap, SMS/WhatsApp metered	Ref-MB
Compute (workers)	Autoscale; idle off-peak	Ref-MB, Netflix-RENO scale policies

5. Industry Comparison

Each system below is summarized as architecture / problems / mistakes / improvements / trade-offs / lessons / copy / avoid.

5.1 Uber — RAMEN → gRPC (real-time push)

- **Architecture.** *Fireball* (decides *when* to push, config-driven, listens to events) → *API gateway* (decides *what* to push; Pull and Push APIs share business logic, Push APIs add a middleware that forwards the pull response to delivery) → *Push delivery system* (persistent connections to apps). Gen-1: Node.js + Ringpop consistent hashing + Redis. Gen-2 (2017): Netty + ZooKeeper + Apache Helix + Redis + Cassandra (Streamgate, StreamgateFE, 5-node Helix controllers). Gen-3 (2019): gRPC + QUIC.
- **Problems faced.** Polling was 80% of gateway traffic; Ringpop's gossip convergence degraded as the ring grew; Node.js single-thread event-loop lag caused topology inconsistency and message loss; SSE was unidirectional, text-only, with delayed (30 s) acks.
- **Mistakes (in hindsight).** Betting the connection layer on decentralized gossip sharding; choosing a text, unidirectional transport that later blocked bidirectional features.
- **What they improved.** Centralized topology via ZooKeeper/Helix (fast failure detection, custom rebalancing) → 99.99% reliability, 1.5M concurrent connections, 250k msgs/s. Then gRPC for instant acks, RTT measurement, multiplexing, binary payloads.

- **Trade-offs.** SSE's simplicity (easy clients in many languages, ran on existing HTTP+JSON stack) vs gRPC's richer capabilities at higher complexity.
- **Lessons.** Separation of concerns is what made the SSE→gRPC swap possible without rewriting business logic; build on industry-standard tech for low maintenance.
- **Copy:** the Fireball/gateway/delivery three-way split; priority buckets; per-message TTL; dedup config; per-user sharding. **Avoid:** decentralized gossip sharding at scale; single-threaded hot paths; text/unidirectional transports for evolving requirements.

5.2 Netflix — RENO

- **Architecture.** *Manhattan* as the single near-real-time event source → priority-based **Amazon SQS** queues → priority-based AWS clusters → Outbound (Netflix messaging system for mobile push; **Zuul Push** persistent connections for web/TV) → **Cassandra** persistent store devices poll from.
- **Problems faced.** Device heterogeneity (smart TVs offline most of the day vs always-on mobiles); 150k events/s peak causing thundering herd; iOS/Android background-activity limits.
- **Mistakes/limits.** Originally scoped to a single use case ("Personalized Experience Refresh"); only later generalized. No guaranteed delivery or batching at time of writing (listed as future work).
- **What they improved.** Hybrid push+pull; staleness filter on event age; notify only **online** devices (via the Zuul registry); aggressive scale-up vs conservative scale-down; event dedup; **bulkheaded** best-effort-per-platform delivery.
- **Trade-offs.** Best-effort-per-platform delivery (resilience) vs guaranteed delivery (not yet built).
- **Lessons.** Make new use cases plug-and-play; shard by event type/priority to scale independently.
- **Copy:** hybrid push+pull, staleness filter, online-only targeting, bulkheaded delivery, priority sharding, poll-store in Cassandra. **Avoid:** assuming one delivery model fits all device types; deferring guaranteed delivery if your domain is transactional (ours is — so we build it in).

5.3 Uber — Mobile Content Delivery (feed)

- **Architecture.** Three content interaction models (urgent / non-urgent / permanent). Supports **both fan-out-on-write and fan-out-on-read** simultaneously. Targeting via Samza rules engine on Kafka (event-based), a real-time matcher (XP-aligned), and offline Hive/Hadoop/Spark jobs (bulk). Cassandra feed store, partitioned by **user_id**, compound key (user_id, item_type, item_id) to avoid write race conditions. JSON template blocks rendered on-device.
- **Lessons / copy.** Fan-out-on-write for real-time + small cohorts; fan-out-on-read where targeting logic is heavy; Cassandra DC replication for global availability; **on-device**

JSON templates so new content types don't require app releases (huge for localization). **Avoid:** baking presentation logic into per-content-type mobile code.

5.4 Uber — ML push timing (Consumer Communications Gateway)

- **Architecture.** Per-user **inbox** (sharded MySQL docstore, partitioned by user-UUID) → *Schedule Generator* (Michelangelo ML + **integer linear program** solver) → *Scheduler* (**Cadence** distributed cron + per-hour Kafka topics, idempotent) → *Push Delivery* (last-mile checks, behind a Kafka buffer). An **XGBoost** model scores $P(\text{order} \mid \text{push}, \text{time})$.
- **Problems solved.** Billions of pushes/month with conflicting messaging, no personalization, 15 hrs/week of manual orchestration.
- **Lessons / copy.** Treat scheduling/frequency-capping as an **optimization problem** with business-rule constraints (expiry, send window, daily cap, min gap); buffer competing messages per user and re-plan whenever a new one arrives; keep the scheduler idempotent. **Avoid:** greedy per-message scheduling that ignores the whole inbox. **Trade-off:** ILP sophistication vs operational complexity — this is a Phase-3 capability, not MVP.

5.5 Slack — Tracing notifications

- **Architecture.** Each notification is its **own trace**, with `notification_id` used as `trace_id`; **span links** tie it to the originating message's (1%-sampled) trace while notifications are **100% sampled**. Spans: trigger → notify → sent → received → opened → read_in_app. Ingested to Elasticsearch/Grafana + data warehouse.
- **Problems solved.** Multi-day, multi-team debugging; lowest-NPS, longest-resolution customer tickets; fragmented per-system log formats.
- **Lessons / copy.** Model the notification lifecycle as a trace with a stable ID; decouple trace context from request context; 100% sampling for notifications; the same trace data doubles as **funnel analytics**. Result: **30% faster triage**. **Avoid:** per-system bespoke logging with no common join key.

5.6 Airbnb — OMNI (promotions & communications)

- **Architecture.** OMNI UI (campaign mgmt, approval, translation, analytics) + backend services: **Audience** (who; ElasticSearch + KV store, 100+ attributes), **Workflow** (when; hourly cron + event-driven, queue-based), **Optimization** (personalization/ranking/send-time), **Presentation** (what/where; validation, translation), **Rendering** (channel-specific), **Delivery** (SQS workers → SendGrid/Twilio/FCM, < 30 s). ~50B comms/yr.
- **Mistakes/regrets.** Content governance, approval, and guardrails were *not* built in from the start; ElasticSearch attribute bloat.
- **Lessons / copy.** A self-service campaign tool with **approval workflows and guardrails from day one**; isolate spiky promotional traffic from regular traffic via elastic scaling +

rate limiting + backpressure; allowlist ES indexed fields and keep schema sparse.

Avoid: unbounded indexed attributes; treating governance as an afterthought.

5.7 Razorpay — webhook notification service scaling

- **Architecture (evolved).** API pods → **priority queues P0/P1/P2** (SQS) → workers → **async DB writes via a Kinesis stream** (DB writer throttles IOPS independently) → data lake. Scheduler re-queues retries. **Rate limiter** + dedicated rate-limit queue. **QoS** lowers priority for customers whose endpoints respond slowly (> 5 min).
- **Problems faced.** Read-query degradation as the table grew; **worker pods couldn't scale because each added DB IOPS**; vertical DB scaling (2x→4x→8x.large) stopped helping; slow customer endpoints blocked workers; spiky loads (sales/IPL/World Cup).
- **Lessons / copy.** **Decouple DB write rate from worker count via a stream** (the single most reusable idea here); priority queues + rate limiting for multi-tenant fairness; QoS to contain misbehaving consumers; "shrink the problem" before re-architecting. **Avoid:** synchronous DB writes on the hot path; vertical DB scaling as a strategy.

5.8 Reference guides (AlgoMaster, Madhur Banger)

- **Copy.** Capacity-estimation method; queue-decoupled pipeline; scheduler table partitioned on `scheduled_time`; storage mix (Postgres + NoSQL + blob); idempotency via Redis `SETNX` + TTL; exponential backoff **with jitter**; DLQ after N retries; per-user partition key for ordering; fallback channels; multi-tenancy via `tenant_id`; provider connection pooling/pre-warming. **Trade-off framing.** Their Kafka-vs-SQS table (low-latency vs low-cost vs high-reliability) is the basis for §10.

5.9 Cross-cutting comparison tables

Transport / connection layer for real-time in-app

System	Choice	Why	Outcome
Uber gen-1	SSE	Simplicity, fit existing HTTP+JSON stack	Worked to 70k QPS but unidirectional, text, delayed acks
Uber gen-2/3	gRPC + QUIC	Bidirectional, binary, instant acks, RTT	1.5M connections, 250k msgs/s
Netflix	Zuul Push (persistent) + push providers	Always-on for web/TV, push for mobile	Hybrid coverage of all device types

Message broker chosen, by company

System	Broker	Why that broker	Lesson for us
Netflix-RENO	Amazon SQS (priority queues)	Managed, simple, priority sharding	SQS is fine when you don't need replay/ordering
Razorpay	SQS + Kinesis stream	SQS for jobs, Kinesis to throttle DB writes	A stream in front of the DB is a powerful decoupler
Uber-Feed/CCG	Kafka (+ Cadence, Cherami)	High throughput, replay, partitioning	Kafka where ordering/replay/volume matter
Ref-MB/AM	Kafka or SQS	Kafka for throughput/replay; SQS for ops-free	Decide by replay + ordering + volume needs
Internal-Draft	Kafka	Replay, retention, per-user ordering, tiered retries	Our backbone

Persistence for delivery history

System	Store	Why
Netflix-RENO	Cassandra	High-scale, devices poll at own cadence
Uber-Feed	Cassandra	DC replication, user-id partitioning, race-free writes
Razorpay	RDBMS + data lake	Audit + analytics, async-written
Ref-AM	Postgres + DynamoDB + S3	Mixed by access pattern
Internal-Draft	Postgres + Redis, Cassandra later	Start simple, add wide-column when log outgrows Postgres

6. Complete Architecture

Every component, and why it exists. The design is the **Internal-Draft** pipeline (two entry points → notification service → Kafka → enrich → route → channel workers → providers) hardened with the service decomposition of **Airbnb-OMNI** and the reliability patterns of **Netflix-RENO** / **Razorpay**.

Layer	Component	Responsibility	Why it exists / evidence
Edge	API Gateway / Load Balancer	TLS, authN, routing, coarse rate-limit, schema validation	Ref-AM; Uber-RAMEN gateway also handles rate-limit/routing/schema validation cross-cutting concerns
Edge	Authentication/Authorization	OAuth2/mTLS, RBAC, per-tenant keys	Ref-AM (OAuth2, RBAC); §18
Ingest	Notification API service	Sync entry: validate, assign idempotency key + notification_id, publish to <code>ingest</code>	Ref-AM, Internal-Draft; Slack-Trace (notification_id minted here = trace_id)
Ingest	Live Alert/Event Engine	Async entry: consume domain-event stream (market ticks), match against <code>alert_subscriptions</code> , generate notifications	Internal-Draft; Uber-RAMEN Fireball; Uber-Feed event targeting
Core	Enrichment service	Resolve contact, locale, tz; check preferences + category; choose channel(s); dedup	Internal-Draft; Netflix-RENO targeting; Ref-MB channel selection
Core	Preference service	Canonical per-channel/category prefs, quiet hours, consent	All sources; fail-open/closed logic from Internal-Draft
Core	Template service	Versioned templates per channel/locale; render placeholders	Ref-AM, Airbnb-OMNI Presentation/Rendering, Uber-Feed JSON blocks
Core	Router	Fan enriched events to per-channel topics	Internal-Draft step 4; Ref-AM step 6 (topic per channel)
Core	Scheduler service	Park future/delayed sends; release when due	Ref-AM, Uber-CCG (Cadence), Internal-Draft (Redis ZSET)
Messaging	Kafka cluster	Durable, partitioned, replayable backbone	Internal-Draft; Uber/Ref consensus

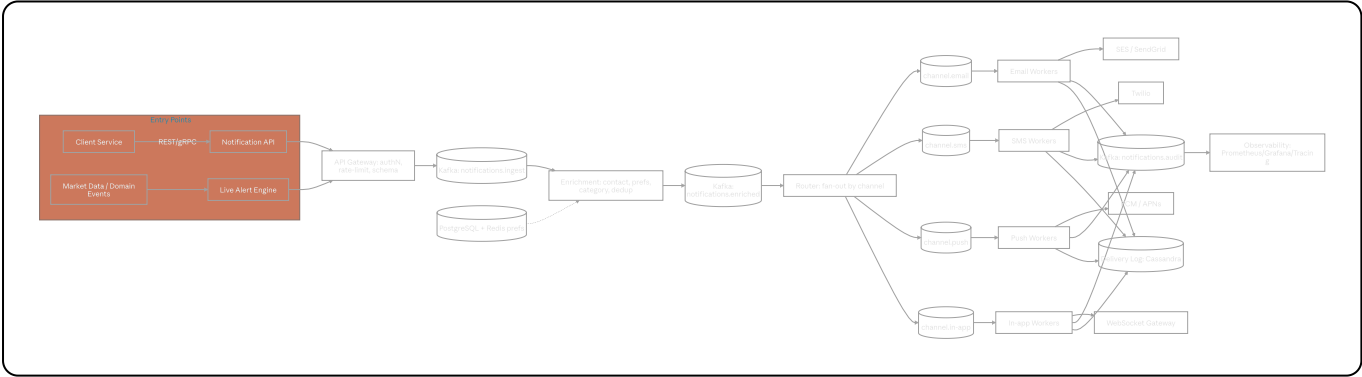
Layer	Component	Responsibility	Why it exists / evidence
Workers	Per-channel worker fleets	Consume channel topic, render, call provider, ack, log	Ref-AM Channel Processors; Airbnb Delivery workers
Channels	Email / SMS / Push / In-app adapters	Provider-specific delivery behind common interface	Airbnb-OMNI Delivery; Ref-MB channel integrations
Channels	WebSocket Gateway	Persistent connections for live in-app + unread	Netflix Zuul Push; Uber-RAMEN; Ref-MB
Reliability	Retry service / tiered retry topics	Delayed re-attempts before DLQ	Internal-Draft (5s/1m/1h); Ref-MB backoff
Reliability	DLQ	Isolate poison messages; manual replay	Internal-Draft, Ref-AM, Ref-MB
Reliability	Rate limiter	Per-user/channel/tenant/event throttles	Razorpay; Ref-AM/MB
Reliability	Circuit breakers	Trip on provider failure; protect workers	Ref-MB; Razorpay QoS
State	PostgreSQL	Contacts, prefs, subscriptions, templates, tokens	Internal-Draft; Ref consensus
State	Redis	Pref cache, idempotency keys, scheduler ZSET, rate limits	Internal-Draft, Ref-MB SETNX
State	Cassandra/ScyllaDB	High-volume delivery log/history at scale	Netflix-RENO, Uber-Feed, Internal-Draft
State	Object storage (S3)	Large attachments, archived logs	Ref-AM
Analytics	ClickHouse + Elasticsearch	Funnel analytics, search, dashboards	Slack-Trace (ES/Grafana + warehouse); Airbnb (ES audience)
Ops	Monitoring (Prometheus/Grafana), Tracing	Metrics, lag, DLQ size, end-to-end traces	Razorpay, Slack-Trace, Internal-Draft

Layer	Component	Responsibility	Why it exists / evidence
Ops	Admin dashboard	Campaigns, approvals, DLQ replay, ops views	Airbnb-OMNI UI; Razorpay Grafana
Ops	Config service + feature flags	Channel/provider config, kill-switches, rollout	Uber-RAMEN config-driven Fireball; Ref-MB
Optional	Audience service	Cohort building from attributes (bulk/campaigns)	Airbnb-OMNI
Optional	Optimization service	Send-time/frequency ML (ILP + XGBoost)	Uber-CCG, Airbnb-OMNI

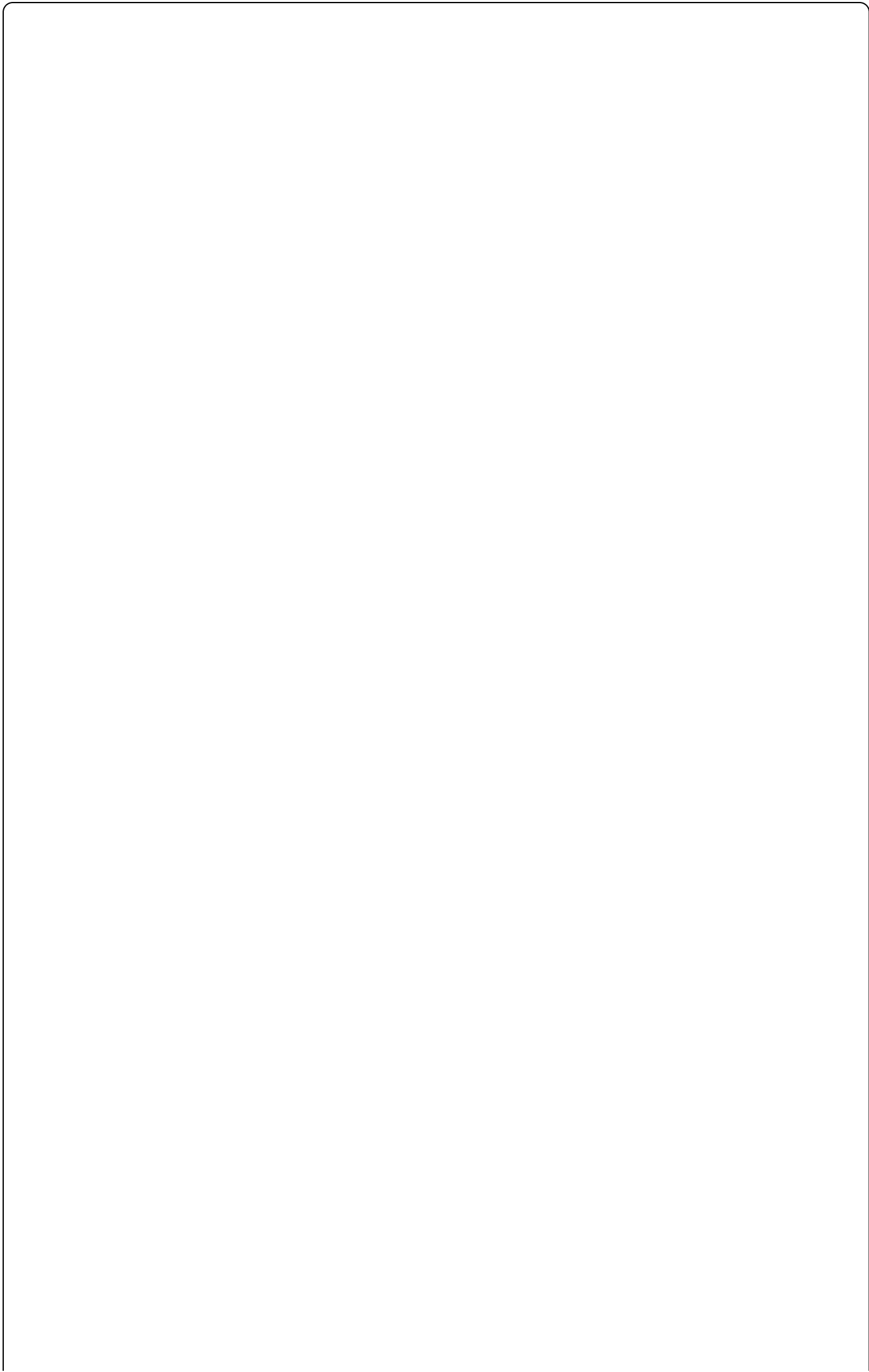
Why the enrichment + router split (not one service): keeping *when/who* (enrichment) separate from *fan-out* (router) is exactly Uber's Fireball-vs-gateway separation and is what let Uber change transports without touching logic. It also lets us scale and tune the two stages independently — the lesson Netflix encodes as priority-specific clusters.

7. Architecture Diagrams (Mermaid)

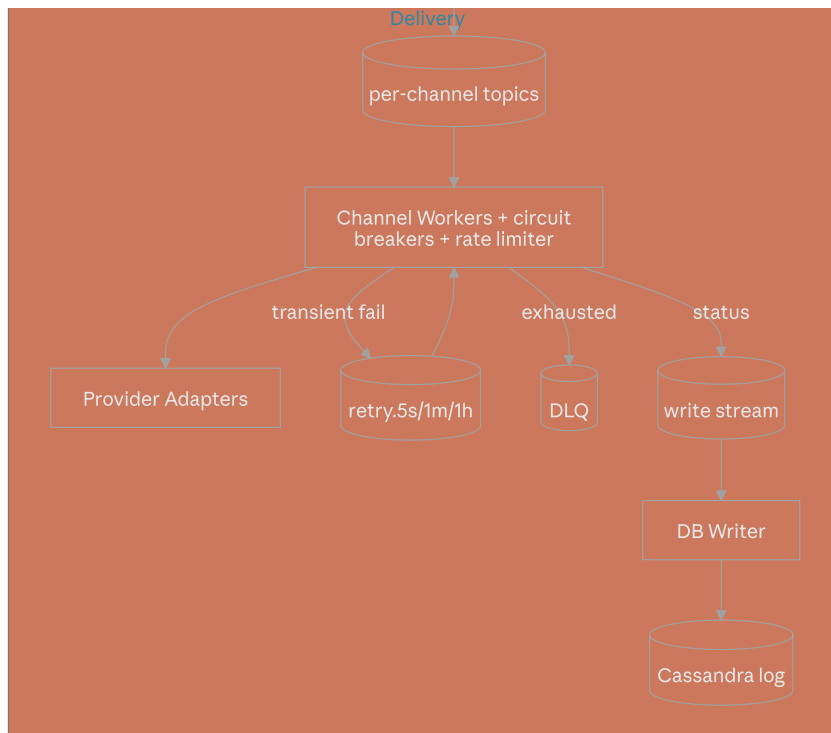
7.1 High-level architecture



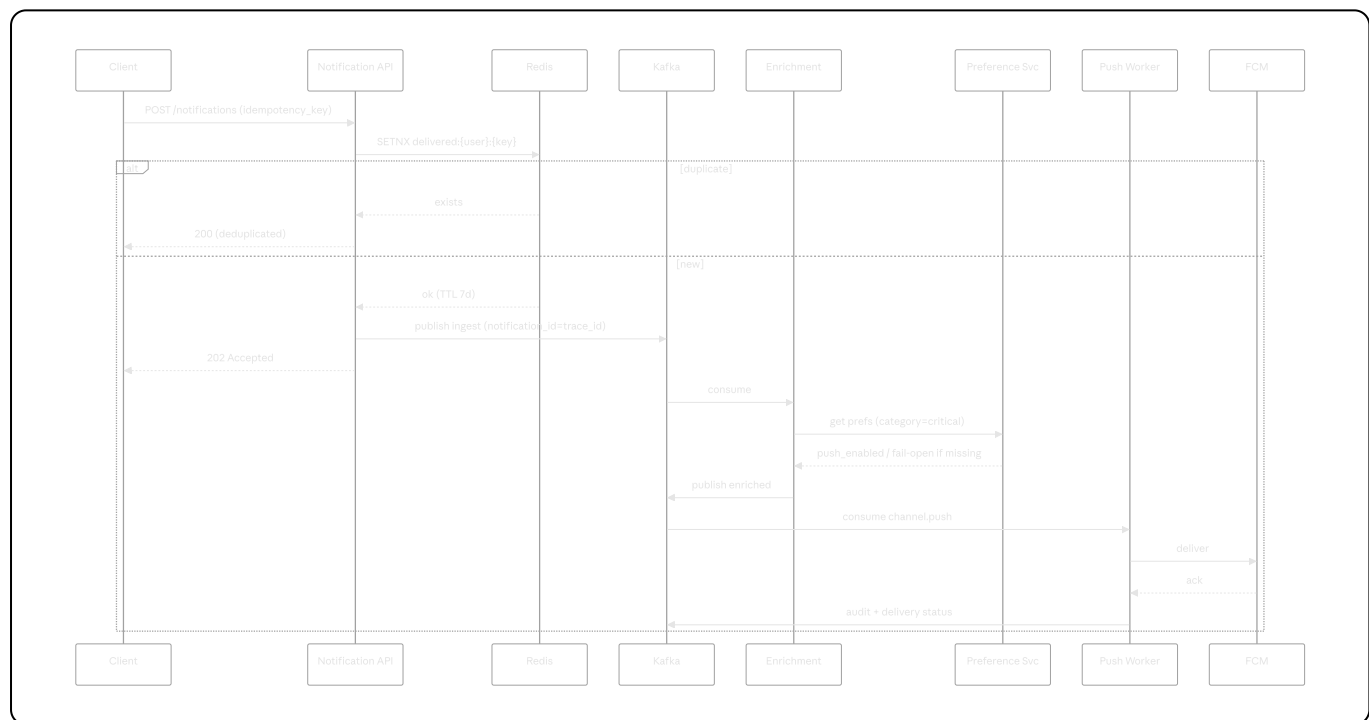
7.2 Detailed component architecture



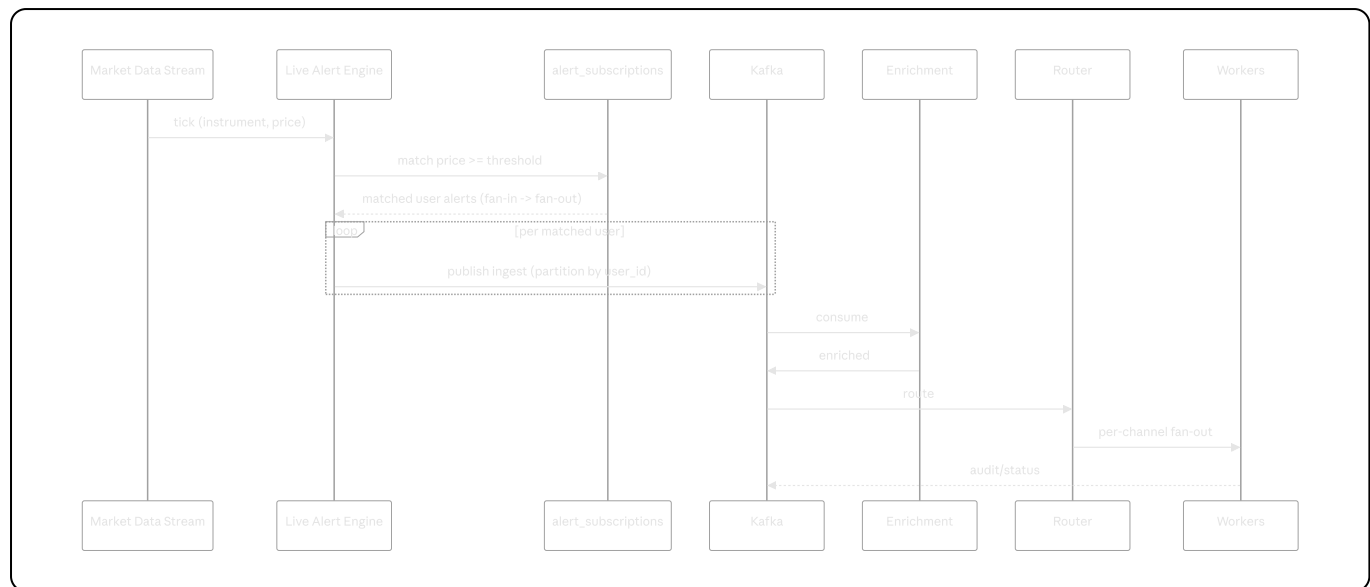




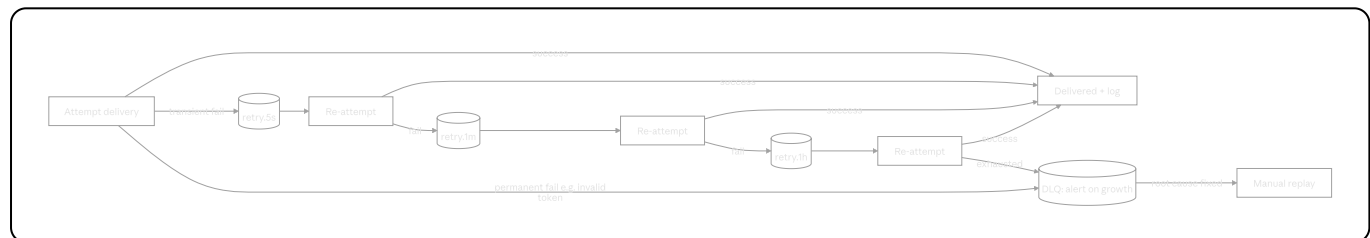
7.3 Sequence — synchronous transactional send (e.g., OTP / order filled)



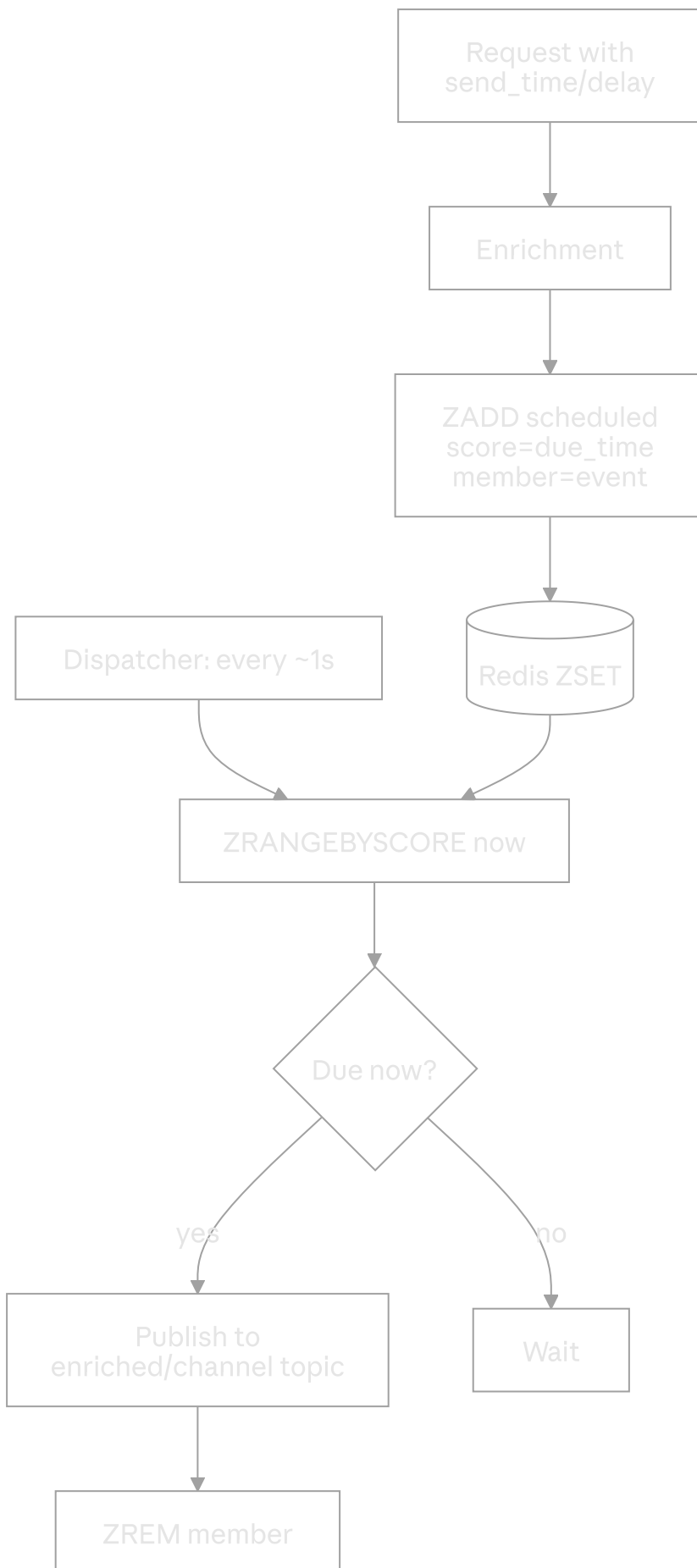
7.4 Sequence — event-driven live alert (market tick → notification)



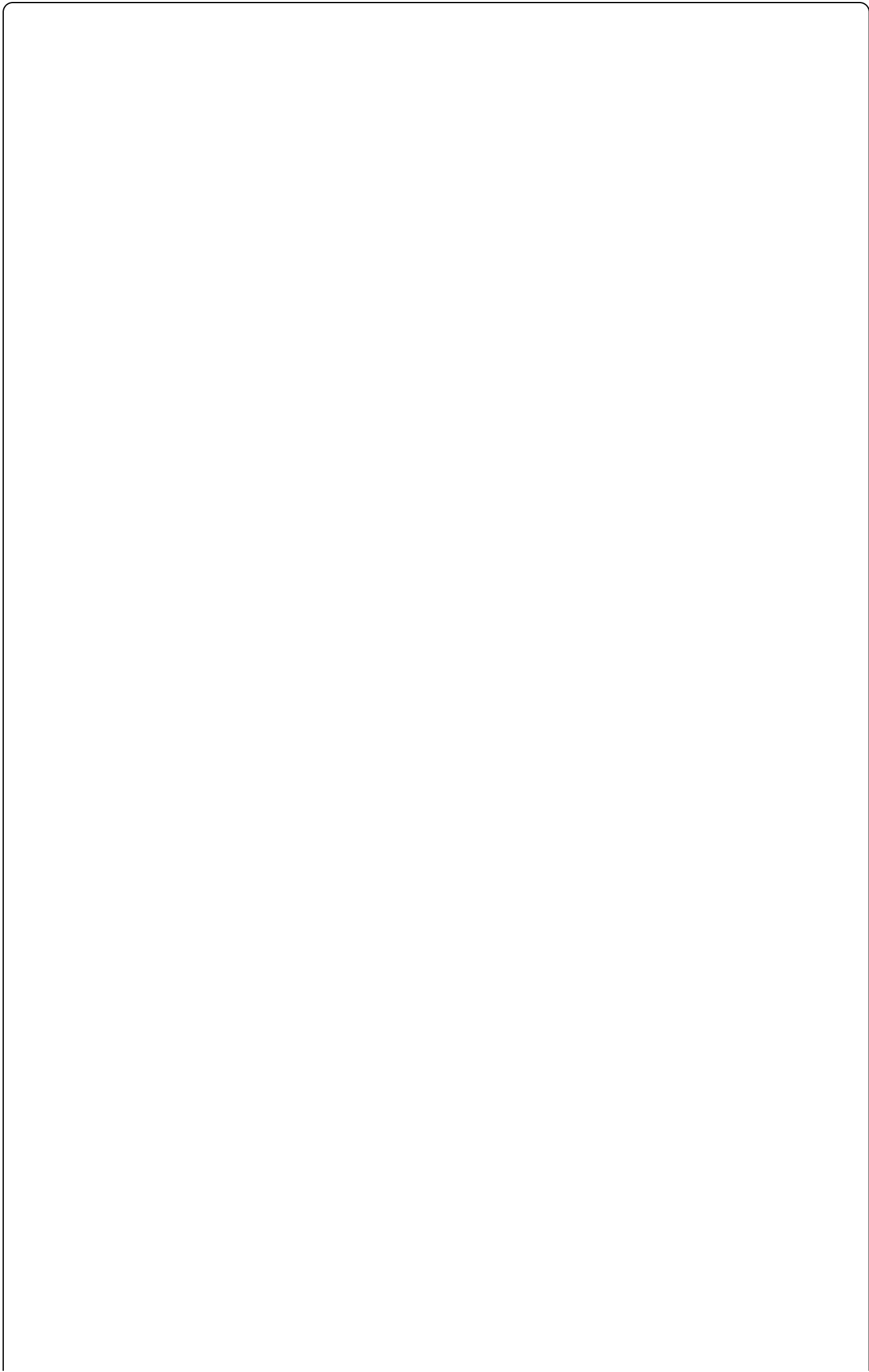
7.5 Retry / failure-handling flow

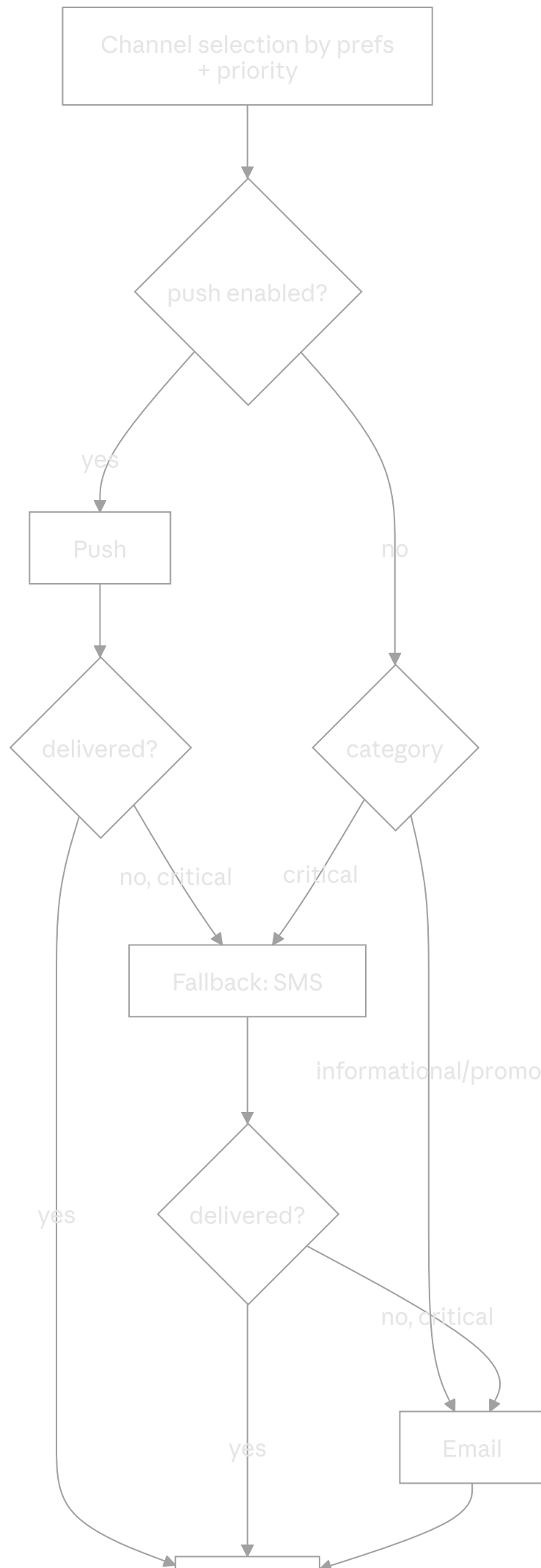


7.6 Scheduling flow (Redis sorted-set timer)



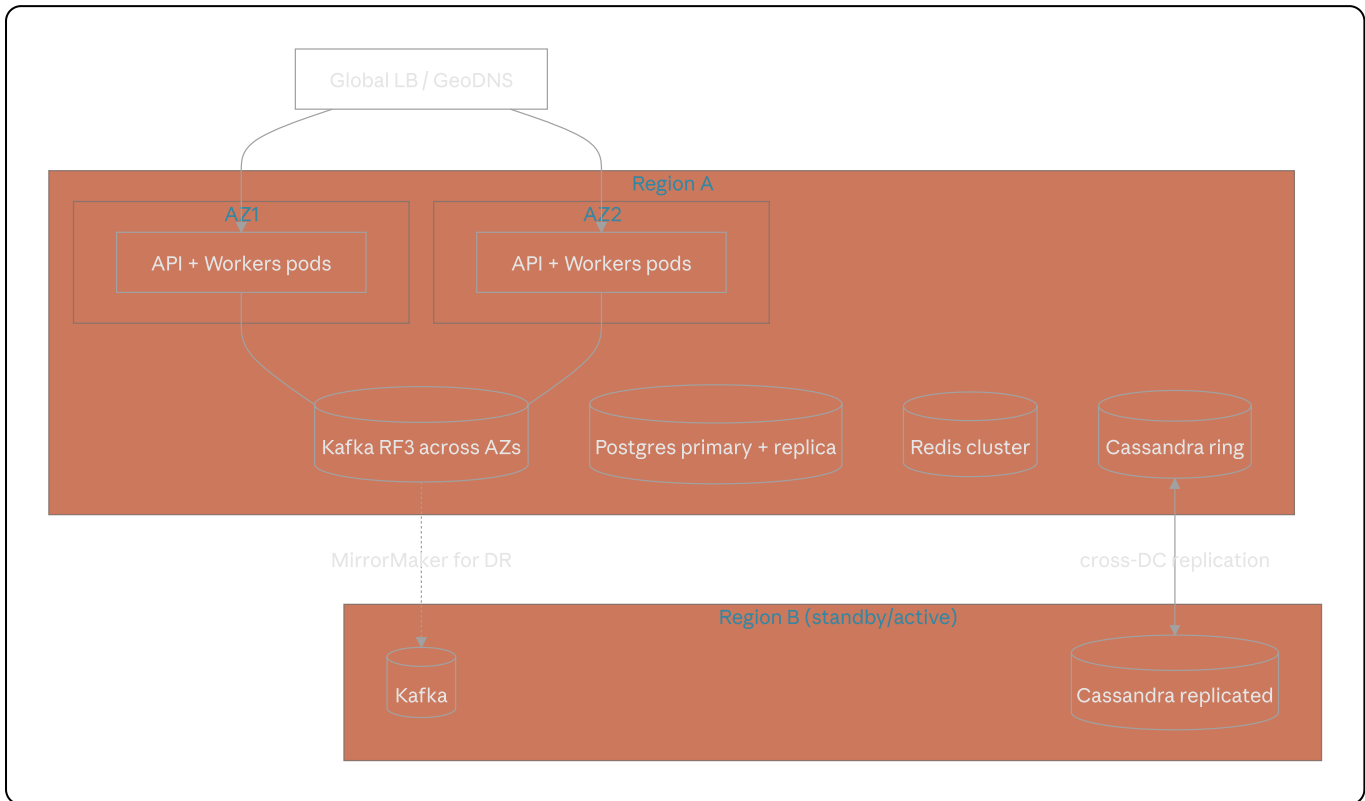
7.7 Multi-channel delivery + fallback



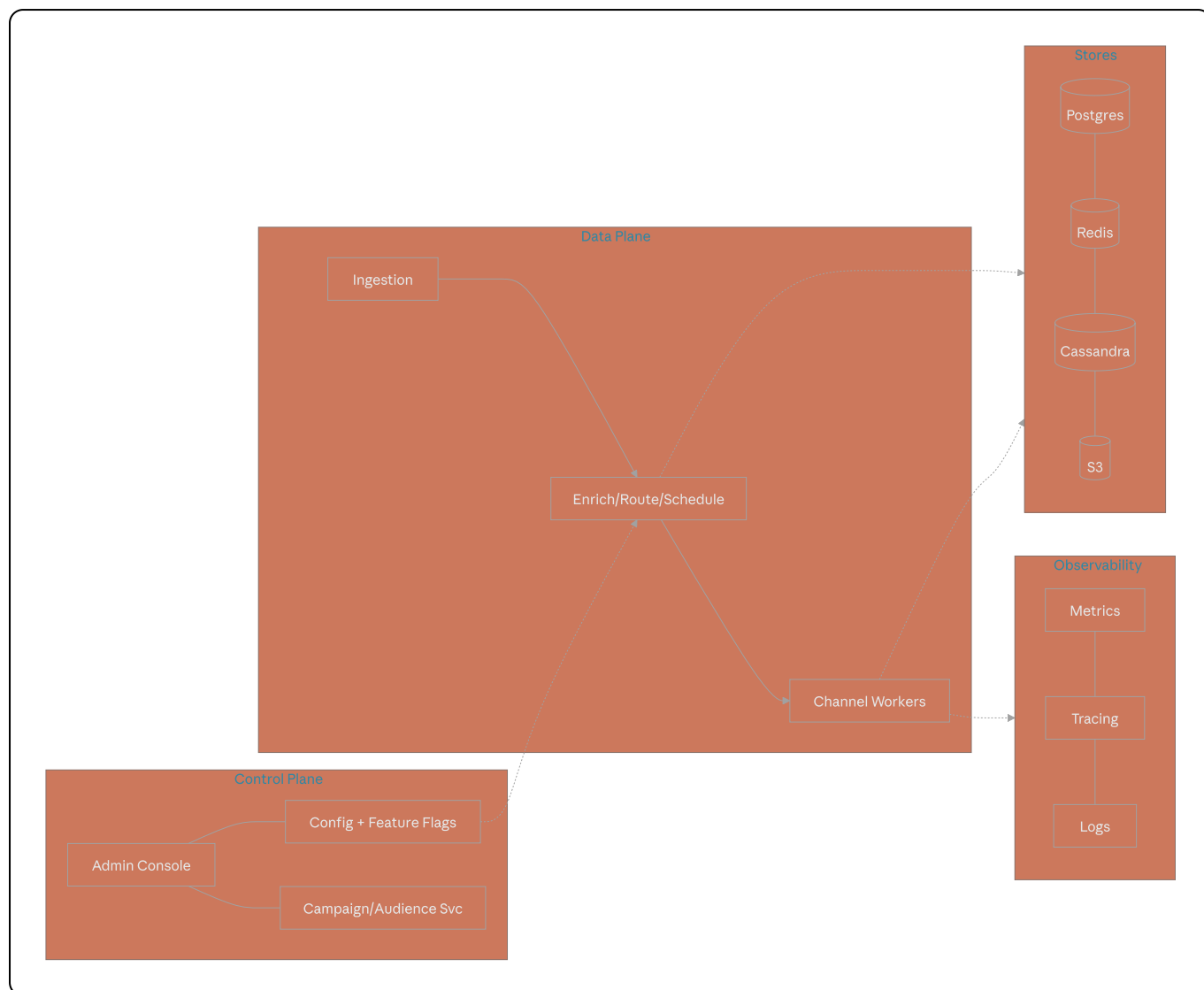


Done

7.8 Deployment (multi-AZ, regional)



7.9 Component diagram (logical ownership)



8. API Design

REST is the primary surface (gRPC offered internally for low-latency, high-volume producers — Uber-CCG used gRPC into its Persistor, and Uber's whole gen-3 moved to gRPC). All endpoints are versioned under `/v1`.

8.1 Conventions

- **AuthN:** OAuth2 client-credentials or mTLS for service-to-service; per-tenant API keys. **AuthZ:** RBAC scopes (`notify:send`, `notify:read`, `admin:campaign`).
- **Idempotency:** clients send `Idempotency-Key` header (or `idempotency_key` in body, per Ref-MB schema). Server dedupes via Redis `SETNX` with 7-day TTL (Ref-MB pattern). Re-sending the same key returns the original result, never a second delivery.
- **Versioning:** URI version + `Accept` header; additive changes don't bump the version.
- **Pagination:** cursor-based (`?limit=50&cursor=...`) for history/log endpoints — avoids deep-offset cost on large tables (relevant to Razorpay's read-degradation problem).

- **Validation:** schema-validated at the gateway *before* publishing to Kafka, so malformed events never enter the pipeline (Internal-Draft "schema validation enforced up front").
- **Rate limits:** surfaced via `X-RateLimit-*` headers and `429` with `Retry-After`.

8.2 Core endpoints

Method	Path	Purpose
POST	<code>/v1/notifications</code>	Send one notification (sync entry)
POST	<code>/v1/notifications:batch</code>	Send up to N in one call (bulk)
GET	<code>/v1/notifications/{id}</code>	Get a notification + delivery status
GET	<code>/v1/users/{user_id}/notifications</code>	Paginated history / inbox
POST	<code>/v1/notifications/{id}:read</code>	Mark in-app notification read
GET/PUT	<code>/v1/users/{user_id}/preferences</code>	Read/update preferences
POST	<code>/v1/users/{user_id}/devices</code>	Register/rotate a push token
GET/POST	<code>/v1/templates</code>	Manage versioned templates
POST	<code>/v1/campaigns</code>	Create a campaign (admin)
POST	<code>/v1/dlq/{id}:replay</code>	Replay a DLQ message (admin)
POST	<code>/v1/webhooks/subscriptions</code>	Subscribe to outbound webhooks

8.3 Example — send (request)

json

```
POST /v1/notifications
Idempotency-Key: user-123-order-789
{
  "event_type": "ORDER_FILLED",
  "priority": "HIGH",
  "category": "transactional",
  "user_id": "user-123",
  "tenant_id": "broker-456",
  "channels": ["PUSH", "SMS"],
  "template_id": "order_filled_v3",
  "locale": "en-IN",
  "data": { "symbol": "INFY", "qty": 10, "price": 1543.20 },
  "send_at": null,
  "idempotency_key": "user-123-order-789"
}
```

8.4 Example — accepted (response)

```
json
HTTP/1.1 202 Accepted
{
  "notification_id": "ntf_01J9...", // also the trace_id (Slack-Trace)
  "status": "ACCEPTED",
  "channels": [
    { "channel": "PUSH", "status": "QUEUED" },
    { "channel": "SMS", "status": "QUEUED" }
  ],
  "deduplicated": false
}
```

8.5 Example — status (response)

```
json
GET /v1/notifications/ntf_01J9...
{
  "notification_id": "ntf_01J9...",
  "user_id": "user-123",
  "attempts": [
    { "channel": "PUSH", "status": "DELIVERED", "attempt": 1, "at": "2026-06-26" },
    { "channel": "SMS", "status": "FAILED", "attempt": 2, "error": "INVALID_NUM" }
  ]
}
```


8.6 Error codes

HTTP	Code	Meaning
400	<code>VALIDATION_ERROR</code>	Missing/invalid fields (rejected pre-Kafka)
401/403	<code>UNAUTHORIZED</code> / <code>FORBIDDEN</code>	Bad creds / missing scope
409	<code>IDEMPOTENCY_CONFLICT</code>	Same key, different payload
422	<code>PREFERENCE_BLOCKED</code>	Suppressed by opt-out (non-critical)
429	<code>RATE_LIMITED</code>	Throttled; see <code>Retry-After</code>
503	<code>DOWNSTREAM_UNAVAILABLE</code>	Provider/circuit open; queued for retry

8.7 gRPC (internal)

proto

```
service Notifier {  
  rpc Send(SendRequest) returns (SendAck);           // unary, low-latency prod  
  rpc SendStream(stream SendRequest) returns (stream SendAck); // high-volume,  
}
```

Used by the Live Alert Engine and other high-QPS internal producers; external clients use REST.

9. Database Design

Principle (Internal-Draft): *don't pick one store religiously — match the store to the access pattern.* Start with **PostgreSQL + Redis**, add a **wide-column store** only when the delivery log outgrows Postgres. This is reinforced by Ref-AM (Postgres + DynamoDB + S3) and the Netflix/Uber choice of Cassandra for high-volume history.

9.1 Store selection and trade-offs

Store	Strengths	Weaknesses	What it holds here	Evidence
PostgreSQL	Relations, multi-way queries, ACID	Write-IOPS ceiling (Razorpay's wall)	Contacts, preferences, subscriptions,	Internal-Draft, Ref-AM, Ref-MB

Store	Strengths	Weaknesses	What it holds here	Evidence
			templates, device tokens	
MySQL	Same class; Uber-CCG used sharded docstore	Same write ceiling	Alternative to Postgres; chosen if shop is MySQL-native	Uber-CCG
Redis	Sub-ms reads, TTL, sorted sets, atomics	In-memory cost; not durable system-of-record	Pref cache, idempotency keys, scheduler ZSET, rate limits	Internal-Draft, Ref-MB
Cassandra/ScyllaDB	Massive writes, time-ordered, DC replication	Limited ad-hoc queries, no joins	Delivery log/history at scale	Netflix-RENO, Uber-Feed, Internal-Draft
DynamoDB	Managed wide-column, autoscale	Cost, query rigidity	Preferences/logs if fully on AWS managed	Ref-AM
ClickHouse	Columnar OLAP, fast aggregations	Not for point lookups	Analytics, funnel, per-channel rollups	[Independent recommendation] (Slack used ES + warehouse)
Elasticsearch	Full-text + attribute search	Operationally heavy (Airbnb's bloat)	Audience segmentation, log search	Airbnb-OMNI, Slack-Trace
S3 / blob	Cheap, durable	High latency	Attachments, archived logs	Ref-AM

Why not Cassandra for everything from day one? Internal-Draft's rule of thumb: the durable long-term record already lives in the compacted Kafka `audit` topic, so a wide-column store is only needed once the *queryable* delivery log outgrows Postgres — typically around the 10M-user line in §4. Adding it earlier is premature operational cost.

9.2 Schemas (relational core — PostgreSQL)

sql

```

-- Who to reach
user_contacts(user_id PK, email, phone, locale, timezone, created_at, updated_at)

-- Per-channel + per-category preferences
notification_preferences(
  user_id PK,
  email_enabled bool, sms_enabled bool, push_enabled bool, inapp_enabled bool,
  marketing_opt_in bool DEFAULT false, -- fail-closed default (Internal)
  quiet_hours jsonb, -- {"start":"22:00","end":"07:00"}
  updated_at timestampz)

-- Push targets: one user -> many devices
device_tokens(id PK, user_id FK, token, platform, active bool, last_seen timestampz)
  -- INDEX (user_id) WHERE active; rotate/disable on FCM 'unregistered' (Ref-MB)

-- User-defined triggers the live engine matches
alert_subscriptions(alert_id PK, user_id FK, instrument, operator, threshold,
  type, status) -- INDEX (instrument, status) for fan-in matching

-- Versioned content
templates(template_id, event_type, channel, locale, subject, body, version,
  PRIMARY KEY(template_id, channel, locale, version))

-- Consent audit (compliance)
consent_log(id PK, user_id, channel, action, source, at) -- append-only

```

9.3 Schemas (high-volume — wide-column)

```

notification_log (PARTITION KEY user_id, CLUSTERING (created_at DESC,
notification_id))
  columns: channel, status, attempts, event_id, error, sent_at, delivered_at
  -- user-id partition => fetch a user's history in one query (Uber-Feed
pattern)
  -- TTL 90 days hot; older rows archived to S3 (Ref-AM archival)

```

Uber-Feed's compound primary key `(user_id, item_type, item_id)` to avoid concurrent-write race conditions is adopted here: simultaneous channel results for the same notification write to **different clustering rows**, never contend.

9.4 Redis structures

```

delivered:{user_id}:{idempotency_key} -> "1"      EX 604800  # dedupe (Ref-MB SETNX)
prefs:{user_id} -> hash                            EX 300    # pref cache
scheduled (ZSET) -> member=event_ref, score=due_epoch  # timer queue

```

(Internal-Draft)

```
rate:{user_id}:{channel}:{window} -> counter      EX window      # rate limit
```

9.5 Indexing, partitioning, sharding, TTL, archival

- **Indexes:** Postgres B-tree on FKs and `((instrument, status))`; partial index on active device tokens.
- **Partition keys:** Kafka by `(user_id)` (ordering); Cassandra by `(user_id)` (point history reads); scheduler table by `(scheduled_time)` (Ref-AM — "partitioned on `scheduled_time` so the system retrieves only notifications in the relevant time range").
- **Sharding:** user-based sharding for relational data (Ref-AM, Ref-MB) — preserves per-user ordering and balances load.
- **TTL:** Redis dedupe 7d; pref cache 5 min; delivery log 90d hot.
- **Archival:** move cold logs to S3/Parquet for cheap long-term analytics (Ref-AM §5.6).
- **Read/write patterns:** writes dominate the delivery log (append-only, async via stream — Razorpay); reads dominate preferences (cache-first) and history (point reads by `user_id`).

10. Messaging System

10.1 Broker comparison

Broker	Ordering	Replay	Consumer groups	DLQ	Best for	Verdict
Kafka	Per-partition	Yes (retention/offsets)	Native	Via topic convention	High throughput, replay, ordering	Backbone
RabbitMQ	Per-queue	No (once consumed)	Via queues	Native DLX	Complex routing, lower volume	Fallback for low volume (Internal-Draft)
Pulsar	Per-partition	Yes (tiered storage)	Native	Native	Kafka-like + multi-tenancy	Strong alt; less ubiquitous
AWS SQS	FIFO queues only	No	N/A (visibility)	Native redrive	Managed, simple	Used by Netflix-RENO,

Broker	Ordering	Replay	Consumer groups	DLQ	Best for	Verdict
						Razorpay, Airbnb
Redis Streams	Per-stream	Limited	Consumer groups	Manual	Lightweight, low latency	Small scale only
NATS (JetStream)	Per-subject	Yes	Yes	Yes	Ultra-low latency edge	Niche

10.2 Recommendation and reasoning

Apache Kafka is the backbone, exactly as in Internal-Draft, because we need all four of: **per-user ordering** (partition by user_id), **replay** (DR + audit + reprocessing), **retention** (tiered retry topics without an external scheduler), and **high throughput** (proven to 250k+ msg/s at Uber).

But the corpus is not unanimous, and the disagreement is instructive:

- **Netflix-RENO, Razorpay, Airbnb-OMNI all chose SQS**, not Kafka. Why? They wanted **managed simplicity and priority sharding** more than replay/ordering. Razorpay even put **Kinesis** (a stream) only in front of the DB, keeping SQS for jobs.
- **The trade-off (from Ref-MB's table):** Kafka wins on throughput/replay/ordering; SQS wins on operational overhead. **Decision rule:** if you need replay + ordering + very high volume → Kafka; if you want ops-free and your volume fits → SQS.
- **Our resolution:** we are transactional + ordered + replay-needing (fintech), so Kafka. We adopt Razorpay's *separate* lesson independently: **put a stream in front of the database** to throttle write IOPS, regardless of the broker.

10.3 Topic taxonomy (from Internal-Draft)

Group	Topic	Role	Retention
Ingestion	<code>notifications.ingest</code>	Raw incoming (API or alert engine)	hours
Ingestion	<code>notifications.enriched</code>	Validated/enriched, ready to route	hours
Delivery	<code>notifications.channel.{email,sms,push,in-app}</code>	Per-channel delivery queues	hours-1d
Retry	<code>notifications.retry.{5s,1m,1h}</code>	Tiered delayed retries, no external scheduler	short

Group	Topic	Role	Retention
Dead-letter	<code>notifications.dlq</code>	Exhausted; isolated for inspection	weeks
Audit	<code>notifications.audit</code>	Durable, log-compacted record of every event	weeks-months

- **Partitioning:** every topic by `(user_id)` for per-user ordering (Internal-Draft, Ref-MB `(user_id % partitions)`).
- **Replication factor** ≥ 3 , producer `(acks=all)`, schema validation at ingestion.
- **Consumer lag, backpressure, exactly-once:** consumer groups parallelize; lag is a primary alert (§17). Kafka transactions/EOS available where a use case truly needs exactly-once, but the default is at-least-once + idempotency (cheaper, simpler — consensus across sources).

10.4 Why tiered retry topics beat a cron scheduler

Internal-Draft's tiered retry topics (`(5s/1m/1h)`) implement delayed retries **inside Kafka** — a message that fails moves to the next tier, and only full exhaustion reaches the DLQ. This avoids a separate scheduler on the retry path and keeps the failing message **off the channel topic** so it can't block healthy traffic (the bulkhead principle from Netflix-RENO and Razorpay's retry-queue segregation). A successful attempt at any tier exits to delivered.

11. Worker Architecture

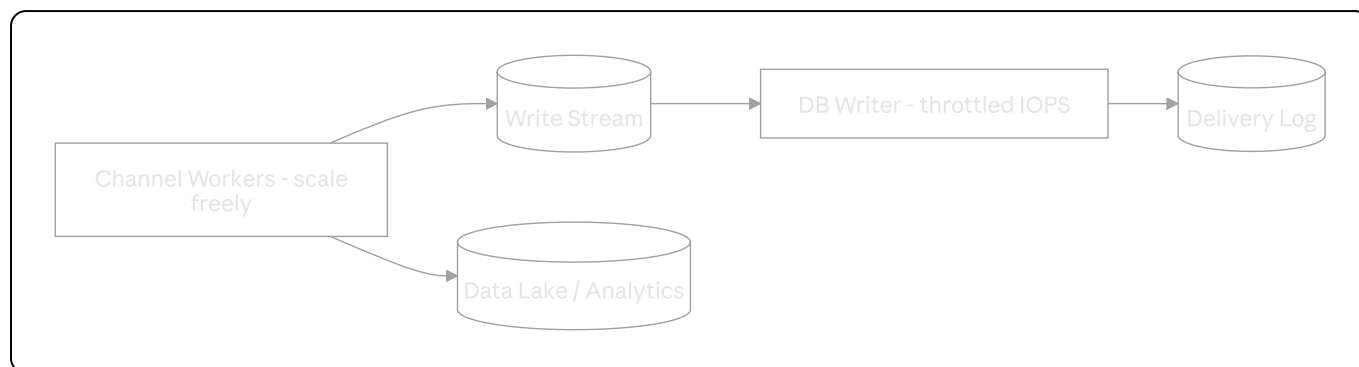
11.1 Pools, concurrency, autoscaling

- **One stateless worker pool per channel** (email/SMS/push/in-app). Channel separation isolates bottlenecks (Ref-MB "push pool separate from SMS pool") and lets each scale on its own lag.
- **Autoscaling** on Kafka consumer lag and CPU via k8s HPA / KEDA. Following **Netflix-RENO**, scale-**up** policies are more aggressive than scale-**down** so compute catches up quickly when queues grow.
- **Concurrency** within a worker is bounded per provider to respect provider rate limits and avoid overwhelming slow endpoints.

11.2 The Razorpay lesson — decouple DB writes from worker count

Razorpay's defining problem: scaling worker pods scaled DB IOPS, and the DB fell over. Their fix is our standard: **workers do not write to the DB synchronously**. They emit delivery status to a **stream**, and a **separate DB-writer** consumes it at a throttled, constant

rate. Consequences (verbatim from their results): DB protected from degradation; workers scale freely; retry queue segregated from main queue; cleaner separation of concerns.



11.3 Retry policy, backoff, poison messages

- **Exponential backoff with jitter** (Ref-MB): $\min(\text{base} \cdot 2^{\text{attempt}}, \text{max}) \cdot \text{jitter}$. Tiered retry topics realize this without a scheduler.
- **Poison messages** (permanent failures — invalid token, bad number, hard bounce) skip retries and go **straight to DLQ**; transient failures (timeouts, 5xx, throttling) walk the retry ladder.
- **DLQ**: alert on growth (Internal-Draft); replay after root-cause fix.

11.4 Circuit breakers & failure isolation

- A breaker per provider trips on sustained error/latency, short-circuiting to retry/fallback instead of blocking workers.
- **QoS for slow consumers** (Razorpay): for webhooks/customer endpoints, if response time exceeds a limit (e.g., 5 min) we **lower that consumer's priority** for a cooldown rather than letting it block the pool — and alert that consumer's stakeholders.
- **Bulkheading** (Netflix-RENO): per-channel/per-platform delivery is best-effort and independent; one provider's outage never blocks others.

11.5 Idempotency & exactly-once strategy

- At-least-once consumption + **Redis dedupe key** ($\text{SETNX delivered:\{user\}:\{key\}}$ EX 7d, Ref-MB) makes delivery **effectively-once**.
- Workers check the key before sending; on success they confirm it; duplicates are acked and dropped.
- Where a use case truly needs exactly-once and the provider supports idempotent sends, we pass the idempotency key through to the provider as well.

12. Notification Pipeline (lifecycle)

End-to-end, the lifecycle is: **receive** → **validate** → **idempotency** → **preference/category**

check → (schedule) → template render → publish → route → worker → provider deliver → ack → status/audit → retry/DLQ → analytics → cleanup. This is the union of Internal-Draft's 7-step flow, Ref-AM's 9 detailed steps, and Airbnb-OMNI's service chain.

sequenceDiagram

```
participant Src as Producer (API / Alert Engine)
participant API
participant Redis
participant Kafka
participant Enr as Enrichment
participant Pref
participant Tpl as Template
participant RT as Router
participant W as Channel Worker
participant Prov as Provider
participant Log as Delivery Log (async)
Src->>API: send / generated event
API->>API: validate (reject malformed pre-Kafka)
API->>Redis: idempotency SETNX
API->>Kafka: ingest (notification_id = trace_id)
Kafka->>Enr: consume
Enr->>Pref: prefs + category (fail-open/closed)
Enr->>Tpl: render template (locale)
Enr->>Kafka: enriched (or park in Redis ZSET if scheduled)
Kafka->>RT: route
RT->>Kafka: per-channel topics
Kafka->>W: consume
W->>Prov: deliver (rate-limited, breaker-guarded)
Prov-->>W: ack / error
W->>Kafka: audit + status
W->>Log: async write (via stream)
W-->>Kafka: retry ladder on transient fail; DLQ on exhaustion
```

Cleanup: short Kafka retention purges ingest/enriched automatically; Redis TTLs expire dedupe/schedule keys; the delivery log ages out to S3 at 90 days. Analytics consume the audit topic continuously (Slack-Trace doubles trace data as funnel analytics).

13. Channel-Specific Design

Common adapter interface (Internal-Draft, Airbnb-OMNI omni-channel principle); each adapter handles provider quirks, retries, and fallbacks.

13.1 Push (FCM / APNs)

- **Providers:** FCM (Android + cross-platform), APNs (iOS); FCM often proxies to APNs (Ref-MB). Netflix uses both behind bulkheaded delivery.
- **Tokens:** store, rotate, invalidate stale tokens daily (Ref-MB "rotate device tokens, daily cleanup of dead tokens"); a `NotRegistered` response disables the token.

- **Payload:** keep small (size limits); metadata for app actions.
- **Latency/cost:** lowest cost, low latency. **Fallback:** for a *critical* push that fails, escalate to SMS (Ref-MB, Internal-Draft).

13.2 Email (SES / SendGrid)

- **Providers:** transactional provider for deliverability + ISP reputation; authenticate domains with **SPF/DKIM/DMARC** (Ref-MB).
- **Content:** HTML or plain text, headers/footers, attachments via S3 (Ref-AM).
- **Failures:** bounces, invalid addresses, spam complaints → suppress + log. Airbnb runs email through the same Delivery Service with vendor abstraction.
- **Cost:** cheap; preferred fallback and bulk/digest channel.

13.3 SMS (Twilio / Nexmo / telecom gateways)

- **Providers:** Twilio/Nexmo; **follow local regulations** (TCPA/DLT in India), long-code vs short-code for throughput/deliverability (Ref-MB).
- **Cost: the dominant variable cost** (Airbnb-OMNI). Apply campaign throttles and channel-shift (move non-critical SMS to push/email) to control spend.
- **Use:** OTPs, security, margin calls (critical/transactional). Character-limit formatting in the adapter (Ref-AM).

13.4 WhatsApp

- **Providers:** WhatsApp Business API via BSPs. **Reachability is geographically limited** (Airbnb-OMNI explicitly calls out WhatsApp/WeChat gaps) — treat as opportunistic, not guaranteed.
- **Constraints:** template pre-approval, 24-hour session windows. Pluggable behind the common interface (P3).

13.5 In-app

- **Persist** notifications so devices can render history + unread counts (Ref-MB, Netflix-RENO poll store). Delivered via WebSocket when online, fetched on app open when not.

13.6 WebSocket gateway (real-time in-app)

- **Lesson from Uber's journey:** Uber went SSE → gRPC because SSE was unidirectional, text-only, with delayed acks. **We use WebSockets (or gRPC streams) — bidirectional, binary-capable, instant acks.** Uber's gen-2 held 1.5M concurrent connections using Netty zero-copy buffers; connection sharding uses **centralized coordination** (ZooKeeper/Helix-style), never decentralized gossip (the Ringpop failure).

- **Hybrid push+pull (Netflix-RENO):** the gateway pushes to online devices (tracked in an online registry like Netflix's Zuul registry) and persists for offline devices to pull — because, as Netflix found, a single delivery model can't serve always-on phones and mostly-off TVs.

13.7 Webhooks (outbound, Razorpay)

- **Subscriber endpoints** receive signed HTTP callbacks on events (`order.paid`-style). Treated as a channel with its own worker pool.
- **QoS for slow subscribers** and per-subscriber rate limits are mandatory — this is exactly the load Razorpay's redesign targeted.

Channel comparison

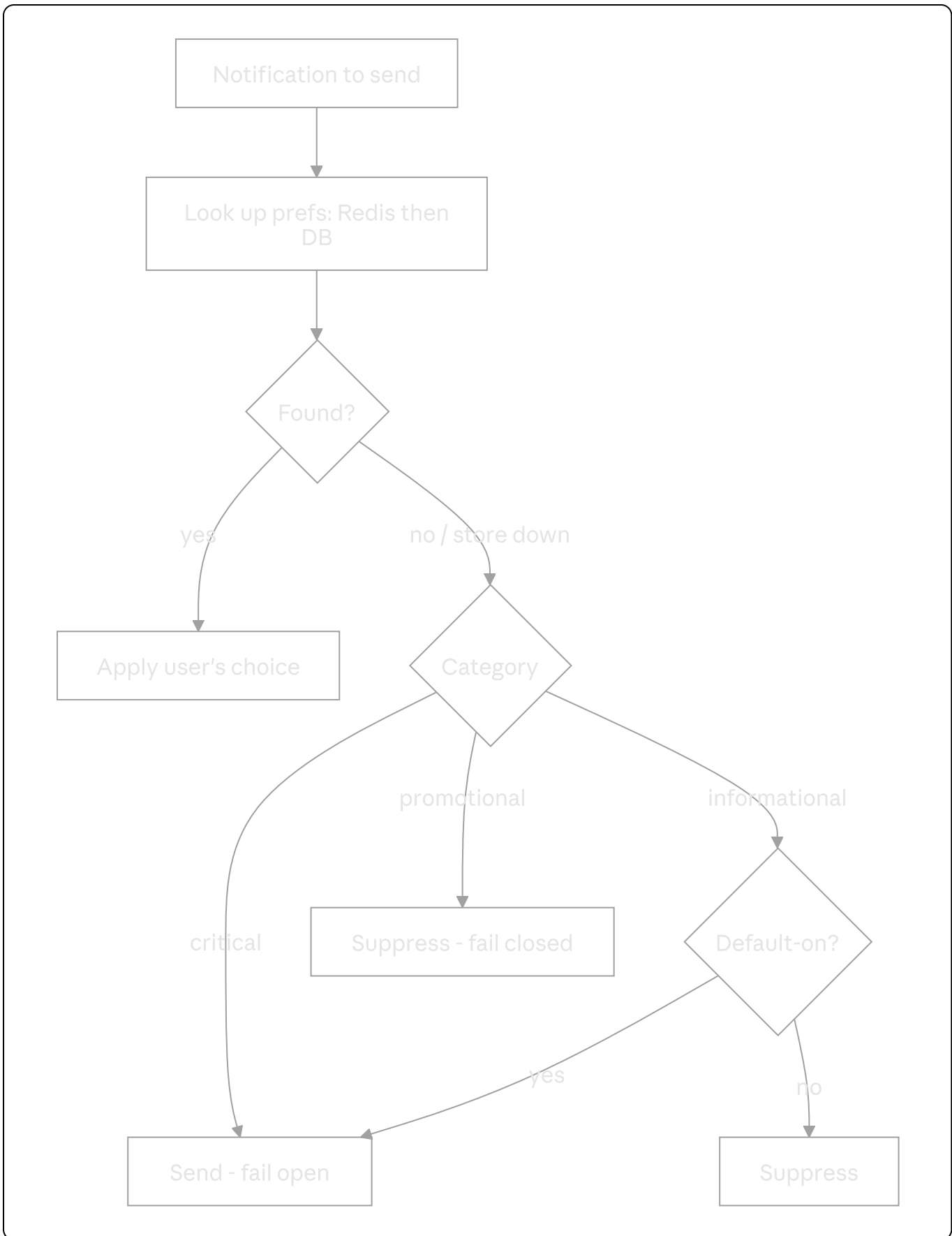
Channel	Latency	Unit cost	Reliability lever	Primary use
Push	Low	Very low	Token hygiene, FCM/APNs bulkhead	Engagement, alerts
In-app/WS	Lowest (online)	Low	Hybrid push+pull, persist	Live updates, unread
Email	Medium	Low	SPF/DKIM/DMARC, reputation	Receipts, digests, fallback
SMS	Low	High	Throttle, channel-shift	OTP, security, margin call
WhatsApp	Medium	Medium	Template approval, sessions	Opportunistic engagement
Webhook	Low	Low	QoS, signing, rate limit	B2B integrations

14. Reliability

Mechanism	How we do it	Evidence
Retries	Tiered Kafka retry topics (5s/1m/1h) + exponential backoff w/ jitter	Internal-Draft, Ref-MB
Circuit breakers	Per-provider breaker trips on error/latency → fallback/retry	Ref-MB, Razorpay QoS

Mechanism	How we do it	Evidence
Timeouts	Bounded per-provider call timeouts; slow ≠ hung	Razorpay (slow customer servers blocked workers)
Rate limiting	Per user/channel/tenant/event; rate-limit queue for breaches	Razorpay, Ref-AM/MB
Duplicate detection	Event dedup at enrichment	Netflix-RENO, Uber-RAMEN
Idempotency	Redis SETNX dedupe key, TTL 7d	Ref-MB, Internal-Draft
Backpressure	Kafka buffers; shed low-priority under overload	Airbnb-OMNI, Ref-MB
Graceful degradation	Drop/defer promotional first; protect critical	Ref-MB, Internal-Draft category logic
Failover	Multi-AZ; Cassandra DC replication; reroute traffic	Uber-Feed, Ref-AM
Regional outages	Active-standby region; Kafka MirrorMaker; GeoDNS	[Independent recommendation] (sources show DC replication; explicit DR topology is added)
Provider outages	Bulkheaded best-effort per channel; fallback channel	Netflix-RENO
Message-loss prevention	RF≥3, acks=all, schema validation, audit topic	Internal-Draft
Recovery	Replay from Kafka retention / audit; DLQ replay	Internal-Draft

Fail-open vs fail-closed (the single most important reliability rule, from Internal-Draft): when preferences are missing or the preference store is briefly unreachable, the *same missing data* yields *opposite* outcomes by category — **critical/transactional → send (fail open)**, **promotional → don't send (fail closed)**, informational → send if default-on, honor explicit opt-out. *Never block an OTP or margin call on a preferences outage; never blast marketing on a guess.*



15. Scaling Strategy

How the system evolves as volume grows, and **which component breaks first at each step** (ordering from §4.3 and Razorpay/Uber experience).

Daily volume	Architecture posture	First bottleneck	Action
10K/day	Single API + 1 worker/channel; Postgres + Redis; even RabbitMQ is fine (Internal-Draft)	None	Keep it simple; don't pre-scale
100K/day	Add Kafka; per-channel topics; autoscale workers	API CPU	Horizontal API instances behind LB
1M/day	Priority queues (P0/P1/P2); rate limiter; pref cache	DB read/write	Cache prefs in Redis; add read replica
10M/day	Async DB writes via stream; add Cassandra delivery log	DB write IOPS (Razorpay's wall)	Decouple writes from workers; introduce wide-column store
100M/day	Priority-specific clusters (Netflix); region-based deployment; bulkheaded channels	Hot partitions; provider limits; WS connection RAM	Repartition; pre-warm provider pools; centralized connection sharding
1B/day	Multi-region active; ML send-time/frequency capping to cut volume; channel-shift	Cost (esp. SMS) + provider throughput	Uber-CCG-style optimization to send <i>fewer, better-timed</i> messages; aggressive channel-shift

Key scaling principles distilled from the corpus:

- Make workers **stateless** and lean on **consumer groups** to parallelize (Ref-MB, Ref-AM).
- **Shard by event type/priority** so traffic classes scale independently (Netflix-RENO).
- **Vertical DB scaling is a dead end** — Razorpay proved 2x→4x→8x.large "was not helping anymore." Re-architect (async writes / wide-column) instead.
- At the very top end, the cheapest scaling is **sending fewer messages** via ML timing + frequency caps (Uber-CCG).

16. Cost Optimization

Lever	Mechanism	Savings rationale	Evidence
Batching	Aggregate digests;	Fewer provider API calls;	Ref-MB (bulk vs real-time),

Lever	Mechanism	Savings rationale	Evidence
	batch provider calls	lower per-message overhead	Netflix-RENO (planned batching)
Channel-shift	Move non-critical SMS → push/email	SMS is the dominant cost	Airbnb-OMNI
ML frequency capping	Send fewer, better-timed messages	Cuts total volume + opt-outs	Uber-CCG
Caching	Redis pref/token cache	Removes repeated DB lookups	Ref-AM, Ref-MB
Async DB writes	Throttle IOPS independent of workers	Avoids buying bigger DBs	Razorpay
Storage tiering	Short Kafka retention; archive logs to S3	Hot storage is expensive	Internal-Draft, Ref-AM
Queue optimization	Right-size partitions; compact audit topic	Less disk, faster recovery	Internal-Draft
Worker autoscaling	Scale to zero off-peak per channel	Pay only for peak when at peak	Ref-MB, Netflix-RENO
Compression	Compress payloads/log records	Less bandwidth/storage	[Independent recommendation]
Connection pooling	Reuse/pre-warm provider connections	Avoids cold-start latency cost	Ref-MB
Dedup	Drop duplicate events early	Avoids paying to send twice	Netflix-RENO, Uber-RAMEN

17. Monitoring & Observability

The headline approach is Slack's: model each notification as its own trace

((notification_id = trace_id)), **100% sampled, span-linked** to the originating event's trace. This is what cut Slack's triage time 30% and what lets the *same* data serve debugging

and funnel analytics. Combine with Razorpay's three-pillars Grafana setup and Internal-Draft's signal list.

17.1 Metrics (Prometheus → Grafana)

- Throughput (notifications/sec) per channel; end-to-end latency (event→delivery); failure rate + retries; **queue/consumer lag**; **DLQ size/growth**; provider success rate; rate-limited-event count; QoS-degraded consumers (Razorpay); WS concurrent connections.

17.2 Tracing

- Spans (Slack model): trigger → notify → sent → received → opened → read_in_app, tagged with notification_id, user_id, channel, status, latency.
- Distributed tracing links producer → queue → worker → provider (Ref-MB).

17.3 Logging

- Structured JSON with event_id/notification_id, user_id, channel, status, latency; centralized (ELK or Loki). Slack's lesson: one common data model/format across all systems, or debugging stays a multi-day cross-team effort.

17.4 Dashboards & alerts

- Alert on: rising consumer lag, growing retry queues, **DLQ spikes** (Internal-Draft); provider failure rate > 5% (Ref-MB); high-response-time customers (Razorpay); success-rate drop; error-budget burn.
- Trend analysis for slow degradations, not just point alerts (Netflix-RENO).

17.5 SLOs & error budgets

- SLOs from §3.1; error budget = 0.01% unavailability/month. Burn-rate alerts gate risky deploys (§20).

18. Security

Area	Control	Evidence
Authentication	OAuth2 client-credentials / mTLS; per-tenant API keys	Ref-AM (OAuth2)

Area	Control	Evidence
Authorization	RBAC scopes on send/read/admin; limit access to critical services	Ref-AM (RBAC)
Encryption	TLS in transit; encryption at rest for PII	Ref-MB
Secrets management	Vault / cloud secret manager; rotate provider keys; revoke on incident	Ref-MB
PII handling	Minimize, encrypt, tokenize phone/email; access-logged	Ref-MB, Airbnb governance
GDPR / right-to-be-forgotten	Delete user data + consent records on request	Ref-MB
Compliance	CAN-SPAM/TCPA (SMS/email), DLT (India), consent audit trail	Ref-MB, Internal-Draft (regulated mandatory messages)
Audit logs	Append-only consent_log + admin action log	Airbnb-OMNI governance
Webhook security	HMAC-sign payloads; verify subscriber TLS; rotate signing keys	Razorpay
API security	Rate-limit at gateway to blunt DoS; schema validation	Ref-AM, Ref-MB
Abuse prevention	Per-tenant/event rate limits; rate-limit queue protected against DOS	Razorpay ("secured the system from different DOS attacks")
Incident posture	On key compromise: revoke provider keys, fail-closed for critical	Ref-MB runbook

19. Testing Strategy

Type	What	Evidence
Unit	Formatting, preference checks, rate-limit logic, backoff math	Ref-MB
Integration	Sandboxed/mock providers; full pipeline through Kafka	Ref-MB

Type	What	Evidence
Load testing	Validate throughput + autoscaling at peak QPS (§4)	Ref-MB, Ref-AM
Stress / soak	Sustained overload to verify backpressure + load-shedding	Airbnb-OMNI (spiky traffic)
Chaos engineering	Block APNs/Twilio; verify fallbacks + bulkheading hold	Ref-MB ("simulate provider outages, ensure fallbacks work")
Failure injection	Kill a broker (verify RF3 no-loss); preference-store outage (verify fail-open/closed)	Internal-Draft
DLQ / idempotency tests	Verify DLQ replay and no-duplicate guarantees	Ref-MB
Performance/benchmark	Per-channel worker throughput; canary perf gates	Ref-MB (canary)

20. Production Deployment

Concern	Approach	Evidence
CI/CD	Automated build/test/deploy; per-service pipelines	Ref-MB
Canary	Roll worker changes to a small % first; gate on metrics	Ref-MB ("canary releases for worker changes")
Blue-Green	For API/gateway; instant rollback	[Independent recommendation]
Rollback	Versioned artifacts; one-command revert; DB migrations reversible	[Independent recommendation]
Database migrations	Expand-contract (add column, backfill, switch, drop); never break in-flight	[Independent recommendation] (informed by Razorpay's "time constraints and regression possibilities")
Feature flags	Channel/provider kill-switches; gradual rollout; config-driven behavior	Uber-RAMEN (config-driven Fireball), Ref-MB

21. Common Mistakes (and how we avoid them)

Each is a real, documented misstep from the sources.

1. **Polling instead of event-driven.** Uber: 80% of gateway requests were polls → battery drain, congestion, cold-start storms. **Avoid:** event-driven push + persisted poll-store.
2. **Decentralized gossip sharding at scale.** Uber's Ringpop convergence degraded as the ring grew. **Avoid:** centralized coordination (ZooKeeper/Helix-class) for stateful connection sharding.
3. **Single-threaded hot path.** Uber's Node.js event-loop lag caused message loss. **Avoid:** keep the delivery hot path off single-threaded runtimes; bound concurrency.
4. **Unidirectional/text transport for evolving needs.** SSE blocked bidirectional features and inflated payloads. **Avoid:** bidirectional binary protocol (gRPC/WebSocket) with instant acks.
5. **Synchronous DB writes on the hot path.** Razorpay: workers couldn't scale without melting the DB. **Avoid:** async writes via a stream.
6. **Vertical DB scaling as a strategy.** Razorpay: 2x→4x→8x.large stopped helping. **Avoid:** re-architect (async + wide-column) instead of buying bigger boxes.
7. **One delivery model for all devices.** Netflix: push-only loses TV notifications. **Avoid:** hybrid push+pull.
8. **No priority isolation.** Without P0/P1/P2 + rate limits (Razorpay) one rogue event/slow consumer starves everyone. **Avoid:** priority queues, rate-limit queue, QoS.
9. **No staleness filter.** Netflix: stale events flood queues. **Avoid:** drop events older than a threshold early.
10. **Governance/guardrails as an afterthought.** Airbnb's top regret. **Avoid:** approval workflows + guardrails from day one.
11. **Unbounded indexed attributes.** Airbnb's Elasticsearch bloated with write-once fields. **Avoid:** allowlist indexed fields, sparse schema, separate batch/realtime.
12. **Fragmented per-system logging.** Slack: multi-day debugging. **Avoid:** one trace per notification, common data model, 100% sampling.
13. **Blocking critical messages on a preferences outage.** **Avoid:** fail-open for critical, fail-closed for marketing.
14. **No idempotency.** Duplicate sends erode trust and cost money. **Avoid:** idempotency keys + Redis dedupe.
15. **Letting failures poison the pipeline.** **Avoid:** tiered retry → DLQ, bulkheaded channels.

22. Final Recommended Tech Stack

One production stack, each choice justified by evidence.

Layer	Choice	Why (with evidence)
Language (services)	Go for workers/hot path; Python for ML/optimization	Go gives multi-core concurrency that avoids Uber's single-threaded Node.js event-loop trap; Razorpay's own blog notes top firms choosing Go. Python for the Uber-CCG-style ILP/XGBoost work.
Language (API)	Go or Java/Kotlin	Uber's high-scale gen-2 was JVM (Netty); either is proven.
API framework	gRPC (internal) + REST gateway	Uber moved to gRPC for instant acks/streaming; REST for external clients (§8).
Message broker	Apache Kafka	Replay, retention, per-user ordering, tiered retries (Internal-Draft). RabbitMQ only if volume is genuinely low.
Stream-before-DB	Kafka (or Kinesis) write stream	Razorpay's IOPS-decoupling lesson.
Primary DB	PostgreSQL	Relational contacts/prefs/subscriptions/templates (Internal-Draft consensus).
Cache / dedupe / scheduler	Redis	Pref cache, idempotency (<code>SETNX</code>), scheduler ZSET, rate limits (Internal-Draft, Ref-MB).
High-volume log/history	Cassandra / ScyllaDB	Netflix + Uber-Feed; user-id partitioned; DC replication. Added at ~10M users.
Object storage	S3	Attachments + archived logs (Ref-AM).
Analytics	ClickHouse + data warehouse	Funnel/throughput rollups (Slack sent trace data to a warehouse).
Search / audience	Elasticsearch	Audience segmentation + log search (Airbnb, Slack) — with allowlisted fields.
Scheduler (heavy/durable)	Temporal/Cadence for complex workflows; Redis ZSET for simple timers	Uber-CCG used Cadence for distributed cron; Internal-Draft uses Redis ZSET for lightweight timers.
Connection sharding	ZooKeeper + Helix-class for WS gateway	Uber's centralized topology beat decentralized Ringpop.
Cluster mgmt	Kubernetes + HPA/KEDA	Stateless autoscaling workers (Ref-MB); aggressive scale-up (Netflix).

Layer	Choice	Why (with evidence)
Container	Docker / OCI	Standard.
CI/CD	GitHub Actions/Argo + canary	Ref-MB canary releases.
Monitoring	Prometheus + Grafana	Razorpay + Internal-Draft.
Tracing	OpenTelemetry, notification-as-trace	Slack model.
Logging	ELK / Loki, structured JSON	Slack common-format lesson.
Cloud	AWS (or equivalent)	Netflix/Airbnb/Razorpay all on AWS-class managed services.
Providers	SES/SendGrid (email), Twilio (SMS), FCM/APNs (push)	Ref-MB, Airbnb, Netflix.
ML platform	XGBoost + ILP solver (later)	Uber-CCG send-time optimization.

23. Implementation Roadmap

Phase	Scope	Key capabilities	Complexity	Dependencies	Milestones
MVP	Core reliable pipeline	F1–F3, F6–F7, F9–F10, F12–F13, F15–F16, F22, F26–F29, F35; Kafka backbone; Postgres+Redis; tiered retry→DLQ; idempotency; fail-open/closed prefs; basic metrics	Medium	Kafka, Postgres, Redis, one provider per channel	Send reliable push retri pref < 5s
Phase 2	Reliability + visibility at scale	F4 (WebSocket), F11/F14/F17(basic)/F18/F19, F24/F25/F26(rich)/F30/F31/F34/F36; async DB writes via stream ; Cassandra delivery log; notification tracing ; priority queues + rate limiter + QoS; analytics dashboards	High	Cassandra, OTel/ClickHouse, WS gateway w/ centralized sharding	10M+ traces repl web app unre

Phase	Scope	Key capabilities	Complexity	Dependencies	Mile
Phase 3	Marketing platform + intelligence	F20-F21, F23, F32-F33; Audience service (ES); campaign UI + approval workflows; multi-tenancy; ML send-time optimization + frequency capping (ILP+XGBoost)	High	ES, ML platform, Temporal/Cadence	Self-campaign govt time redu cost
Phase 4	Global scale + DR	Multi-region active; Kafka MirrorMaker DR; region-based clusters; channel-shift cost engine; chaos program	High	Multi-region infra	100l RTC doct chac

Engineering-complexity notes: the highest-risk items are the WebSocket gateway connection-sharding (Uber spent multiple generations here — do not under-budget it) and the async-write + wide-column migration (Razorpay's regression risk). Both are Phase-2 and should each get a dedicated design review.

24. Final Conclusion

If I had to build this today for a company sending millions of notifications daily, I would build the Internal-Draft pipeline, hardened with the operational lessons of Razorpay, Netflix, Slack, and Airbnb — and I would build it event-driven from line one.

Concretely: an **event-driven, Kafka-backed pipeline** with **two entry points** (sync API + a domain-event/alert engine), a clean **separation of triggering, enrichment/routing, and delivery** (Uber's most durable architectural lesson — it's why they swapped transports twice without rewriting logic), **per-channel stateless worker fleets** behind a common adapter interface, and **PostgreSQL + Redis** as the system of record, adding **Cassandra** for the delivery log only when it outgrows Postgres.

The reliability spine is non-negotiable and entirely evidence-backed: **at-least-once + idempotency keys** for effectively-once delivery; **tiered retry topics → DLQ** so failures never poison the pipeline; **priority queues + rate limiting + QoS** so a margin call never waits behind a marketing blast and one slow consumer never blocks the fleet (Razorpay); **bulkheaded per-channel delivery** so one provider outage is contained (Netflix); **fail-open for critical / fail-closed for promotional** so a preferences outage never drops an OTP and never spams on a guess (Internal-Draft); and **async DB writes through a stream** so the datastore — historically the first thing to fall over — is decoupled from worker scaling (Razorpay).

I would treat **observability as a feature, not an add-on**, modeling each notification as its own 100%-sampled trace (Slack) from day one, because the alternative is multi-day debugging and the worst customer-support tickets you'll ever own. And I would defer the

genuinely hard, genuinely optional pieces — ML send-time optimization (Uber-CCG), self-service campaigns with governance (Airbnb), multi-region DR — to later phases, while making sure the MVP's seams (topic taxonomy, idempotency, preference categories) already anticipate them.

The single thread running through every one of these systems: **decouple aggressively, isolate failures, never trust a provider or a downstream to be fast or up, and design so that the cheapest, safest default — drop a duplicate, defer a promo, retry a transient, send the OTP anyway — is also the automatic one.** That is the system worth building.

*End of design proposal. Diagrams are Mermaid and render inline in compatible viewers. Recommendations tagged [**Independent recommendation**] are supplied from general distributed-systems practice where the source material was silent.*